A Project Report

On

“LIVE EVENT”

Submitted to the

Department of Computer Science

In partial fulfilment of the

MASTER OF COMPUTER APPLICATIONS

Under the guidance of

MS. SIJI P JOY

Project Done by

ASHIK E A

(Reg No: 24222943)

DEPARTMENT OF COMPUTER SCIENCE



(Affiliated to Mahatma Gandhi University, Kottayam)

March-2026

NIRMALA COLLEGE

MUVATTUPUZHA AUTONOMOUS

CERTIFICATE

This is to certify that the project entitled

"LIVE EVENT"

has been successfully carried out by

ASHIK E A

(Reg No: 24222943)

In partial fulfilment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

of Mahatma Gandhi University, Kottayam during the period 2025-2026

Ms. Siji P Joy
INTERNAL GUIDE

Ms. Sherry O Panicker
HEAD OF THE DEPARTMENT

Place: Muvattupuzha

Date:

NIRMALA COLLEGE

MUVATTUPUZHA AUTONOMOUS

CERTIFICATE

This is to certify that the project entitled

“LIVE EVENT”

has been successfully carried out by

ASHIK E A

(Reg No: 24222943)

In partial fulfillment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

of Mahatma Gandhi University, Kottayam during the period 2025-2026

Ms. Siji P Joy

INTERNAL GUIDE

Place: Muvattupuzha

Date:



NIRMALA COLLEGE

MUVATTUPUZHA AUTONOMOUS

DECLARATION

I, ASHIK E A , hereby declare that the project work entitled "**LIVE EVENT**" is an authenticated work carried out under the guidance of **Ms. SIJI P JOY** for the partial fulfilment of the course **MASTER OF COMPUTER APPLICATIONS**. This work has not been submitted for similar purpose anywhere else except to **NIRMALA COLLEGE, MUVATTUPUZHA (AUTONOMOUS)**.

I understand that detection of any such copying is liable to be punished in any way the school deems fit**.**

ASHIK E A

Place: **Signature**

Date:

ACKNOWLEDGEMENT

At the outset, I thank God Almighty for making endeavour a success

I express my gratitude to **Rev. Dr. Jestin K. Kuriakose** Principal, of **NIRMALA COLLEGE, MUVATTUPUZHA (AUTONOMOUS)**, for providing me with adequate facilities, ways and means by which I are able to complete the project work.

I express my sincere thanks to **Ms. SHERRY O PANICKER** Head. Department of Computer Science, and my project guide **Ms. SIJI P JOY** who has been showing deep interest in my project and inspired me through development by valuable suggestion, and all the faculty members of the department of Computer Science for their sincere help and support.

Last but not the least, I also express my profound gratitude to all other well- wishers who assisted me in various occasions during the project work.

ASHIK E A

Table of Contents

Chapter 1 - Introduction 1
1.1 Introduction 2
1.2 Problem Statement 2
1.3 Scope and Relevance of the Project 3
1.4 Objectives 3
Chapter 2 – System Analysis 5
2.1 Introduction 6
2.2 Existing System 6
2.2.1 Limitations of Existing System 7
2.3 Proposed System 8
2.3.1 Advantages of the Proposed System 8
2.4 Feasibility Study 9
2.4.1 Technical Feasibility 10
2.4.2 Operational Feasibility 10
2.4.3 Economic Feasibility 11
2.5 Software Engineering Paradigm Applied 11
Chapter 3 – System Design 13
3.1 Introduction 14
3.2 Database Design 14
3.2.1 Entity Relationship Model 17
3.2.2 Data Dictionary 19
3.3 Object Oriented Design-Uml Diagram 27
3.3.1 Usecase Diagram 27
3.3.2 Sequence Diagram 29
3.3.3 Activity Diagram 32
3.4 Modular Design 35
3.4.1 Modules Description 37
3.5 Input Design 42
3.6 Output Design 50
Chapter 4 – System Environment 56
4.1 Introduction 57
4.2 Software Requirements Specification 57
4.4 Tools And Platforms 58

4.4.1 Front End Tool 58
4.4.2 Back End Tool 59
4.4.3 Operating System 62
Chapter 5 – System Implementation 63
5.1 Introduction 64
5.2 Coding 64
5.2.1 Coding Standards 65
5.2.2 Sample Codes 65
5.3 Unit Testing 72
5.3.1 Test Plan 73
5.3.2 Test Cases 76
Chapter 6 – System Testing 79
6.1 Introduction 80
6.2 Integration Testing 81
6.3 System Testing 82
6.3.1 Test Plan & Test Cases 83
Chapter 7 – System Maintenance 84
7.1 Introduction 85
7.2 Maintenance 85
Chapter 8 – System Security Measures 89
8.1 Introduction 90
8.2 Database Level Security 90
8.3 System-Level Security 91
Chapter 9 – System Planning and Scheduling 93
9.1 Introduction 94
9.2 Planning a Software Project 94
9.2.1 Steps Involved in Planning a System 95
Chapter 10 – Future Enhancement and Scope of Further Development 98
10.1 Introduction 99
10.2 Merits of the System 99
10.3 Limitations of the System 100
10.4 Future Enhancement of the System 100
11 Annexure: 102
11.1 Organization Profile 103
11.2 Document Glossary, Figures, Tables 103

11.3 References 105
11.3.1 Books 105
11.3.2 Websites 105

ABSTRACT

Event management plays a crucial role in organizing conferences, seminars, workshops, cultural programs, and other social or professional gatherings. Traditional event management methods often involve manual registration processes, paper-based tickets, and limited communication between event organizers and participants. These manual processes can lead to inefficiencies such as registration errors, ticket duplication, difficulty in tracking participants, and challenges in managing event data effectively. Additionally, event organizers often face difficulties in analyzing participant feedback and managing attendance during the event. In the digital era, there is a strong need for an automated and centralized platform that simplifies event planning, registration, ticket management, and communication between organizers and attendees.

The **LiveEvent – Event Management System** is a web-based application designed to simplify and automate the process of organizing and managing events. The system provides a centralized platform where users can easily browse available events, register for events, and receive digital tickets through an online interface. Event organizers can create and manage events efficiently using a dedicated dashboard that allows them to update event details, monitor registrations, and analyze participant data. The system is developed using modern web technologies such as **Python, Django Framework, HTML, CSS, and JavaScript**, which ensure a reliable and user-friendly environment for event management.

One of the key features of the LiveEvent system is the implementation of **QR code-based ticket generation**. When a user successfully registers for an event, the system generates a unique QR code ticket that can be used for event entry verification. This feature helps prevent ticket duplication and improves the efficiency of the event check-in process. The system also supports **email-based ticket delivery**, allowing users to receive their tickets directly in their email along with important event details. Additionally, the system integrates calendar functionality using **ICS files**, enabling participants to add events directly to their digital calendars and receive reminders before the event date.

The system is designed with multiple user roles to ensure efficient management and control. The **User Module** allows participants to register, browse events, book tickets, and provide feedback after attending an event. The **Event Organizer Module** enables organizers to create events, manage event details, monitor participant registrations, and analyze event performance. The **Admin Module** acts as the central authority responsible for managing users, organizers, and events within the system. The admin can also monitor feedback submitted by participants and respond to it when necessary.

Another important feature of the system is the **Feedback and Sentiment Analysis Module**. After attending an event, users can submit feedback about their experience. The system analyzes the feedback using sentiment analysis techniques to determine whether the feedback is positive, negative, or neutral. This helps event organizers understand participant satisfaction and identify areas for improvement in future events.

The LiveEvent system also provides reporting and analytics features that allow organizers to view event statistics, participant lists, and attendance reports. These insights help organizers evaluate the success of events and make better decisions for future event planning. By automating the entire event management process, the system reduces manual work, improves accuracy, and enhances the overall experience for both organizers and participants.

In conclusion, the **LiveEvent – Event Management System** provides a comprehensive solution for managing events efficiently in a digital environment. The system simplifies event registration, ticket management, attendance verification, and feedback analysis through a centralized web platform. By integrating modern technologies and automation features, the LiveEvent system improves event organization, enhances participant engagement, and ensures better management of event-related activities.

Chapter 1 - Introduction

1.1 Introduction

Event management plays an important role in organizing various activities such as seminars, workshops, conferences, and cultural programs in organizations and educational institutions. Managing these events effectively requires proper coordination between event organizers and participants. Traditionally, event registration and ticket management were handled manually using paper forms and physical records. These methods often lead to errors, delays, and difficulties in maintaining participant information.

With the advancement of web technologies, digital event management systems have become an efficient way to organize and manage events. These systems allow users to register for events online while enabling organizers to manage event details, participants, and schedules in a centralized platform.

The **LiveEvent – Event Management System** is a web-based application developed to simplify the process of event management. The system allows users to browse available events, register for events, and receive digital tickets. Event organizers can create and manage events, monitor registrations, and track event performance. The system also includes features such as **QR code ticket generation, email notifications, and feedback collection**, which help improve the efficiency of event management and enhance the overall user experience.

1.2 Problem Statement

Many organizations still rely on traditional methods for managing events, which can create several difficulties. Manual registration processes, paper-based tickets, and separate communication channels often lead to errors, inefficiencies, and difficulty in managing participant information. These methods also make it challenging for organizers to track event registrations and maintain accurate records.

Another major issue is the lack of a centralized platform where users can easily view upcoming events and register for them online. Participants may miss important updates or event details due to ineffective communication systems. Event entry verification is also difficult when tickets are managed manually, which may lead to duplication or unauthorized entry.

Therefore, there is a need for a digital system that can automate event registration, ticket management, and communication between organizers and participants. The **LiveEvent - Event Management System** aims to solve these problems by providing a centralized web-based platform that simplifies event management and improves efficiency for both organizers and attendees.

1.3 Scope and Relevance of the Project

The **LiveEvent – Event Management System** is designed to provide a centralized platform for managing and organizing events efficiently. The system allows users to browse upcoming events, register for events online, and receive digital tickets. Event organizers can create events, update event details, manage participants, and monitor event activities through an organized dashboard.

The scope of this project includes simplifying the event registration process, reducing manual work, and improving communication between organizers and participants. The system provides features such as QR code-based ticket generation, email notifications, and event reminders, which help improve the efficiency of event coordination and management.

The relevance of this project lies in the increasing demand for digital solutions in event management. Many organizations, educational institutions, and businesses regularly conduct events and require an efficient system to manage registrations, tickets, and participant data. By providing an automated and user-friendly platform, the LiveEvent system helps improve the overall event management process while ensuring better organization and accessibility.

1.4 Objectives

The main objective of the **LiveEvent – Event Management System** is to develop a web-based platform that simplifies and improves the process of organizing and managing events. The system aims to provide an easy and convenient way for users to browse upcoming events and register for them online. It also allows event organizers to create, update, and manage events efficiently through a dedicated interface. Another objective of the system is to generate digital tickets with QR codes to ensure secure and quick verification during event entry. The system also focuses on improving communication between organizers and participants by sending notifications and event updates. Additionally, the platform provides a feedback mechanism that allows users to

share their experience after attending an event, helping organizers evaluate and improve future events. Overall, the objective of the LiveEvent system is to reduce manual work, improve accuracy, and provide an efficient and user-friendly solution for event management.

The main objectives of the **LiveEvent – Event Management System** are:

- To develop a web-based platform for efficient event management.
- To allow users to browse available events and register for them online.
- To enable event organizers to create, update, and manage events easily.
- To generate digital tickets with QR codes for secure event entry verification.
- To send automated notifications and reminders about events to participants.
- To provide a feedback system for users to share their experience after attending events.
- To allow administrators to manage users, organizers, and events within the system.
- To improve the efficiency and accuracy of event registration and ticket management.

Chapter 2 – System Analysis

2.1 Introduction

System analysis is an important phase in the development of any software system. It involves studying the existing system, identifying its limitations, and understanding the requirements of the new system. The main purpose of system analysis is to determine how a system should function in order to meet user needs effectively and efficiently. During this phase, detailed information about the current processes, problems, and user expectations is collected and analyzed.

In the context of the **LiveEvent – Event Management System**, system analysis focuses on understanding the problems associated with traditional event management methods and identifying the requirements for a more efficient digital solution. The analysis helps in examining how event registration, ticket management, participant tracking, and feedback collection are currently handled and how these processes can be improved using a web-based system.

This phase also involves evaluating the feasibility of the proposed system and identifying the technologies and tools required for implementation. By carefully analyzing the requirements and limitations of the existing system, the LiveEvent platform is designed to provide a more organized, efficient, and user-friendly solution for managing events.

2.2 Existing System

In many organizations and institutions, event management is still handled using traditional or semi-manual methods. Event announcements are usually shared through posters, emails, or social media, and participants often need to register using manual forms or separate online tools. Event organizers typically maintain participant records using spreadsheets or manual documentation.

Ticket distribution and participant verification are also handled manually in many cases. Organizers may issue printed tickets or maintain participant lists, which can create confusion during event entry. Communication between organizers and participants is often carried out through multiple platforms, making it difficult to keep track of important updates and event information.

These traditional systems lack a centralized platform that integrates event creation, registration, ticket management, and feedback collection. As a result, managing events becomes more complex and time-consuming, especially when dealing with a large number of participants.

2.2.1 Limitations of Existing System

The existing event management system has several limitations that affect efficiency and organization. Some of the major limitations are:

- **Manual registration process**, which can lead to data entry errors and duplication.
- **Lack of a centralized system** for managing event details and participant information.
- **Difficulty in tracking registrations** and maintaining accurate records of attendees.
- **Manual ticket distribution**, which increases the risk of ticket loss or duplication.
- **Inefficient entry verification**, leading to delays during event check-in.
- **Limited communication channels** between organizers and participants.
- **Difficulty in updating event information** once registration has started.
- **Time-consuming management process**, especially for large events with many participants.
- **No automated reminder system** for notifying participants about upcoming events.
- **Limited or no feedback collection mechanism** after the event.
- **Difficulty in analysing participant responses and event performance.**
- **Higher chances of human errors** in maintaining records and participant lists.
- **Lack of proper reporting tools** for organizers to evaluate event success.

- **Dependence on multiple tools** such as spreadsheets, emails, and manual records.
- **Security risks** due to improper handling of participant data.

2.3 Proposed System

The **LiveEvent – Event Management System** is proposed to overcome the limitations of the existing manual event management methods. The system provides a centralized web-based platform where users can easily browse upcoming events, register for events online, and receive digital tickets. This platform simplifies the entire event management process by automating registration, ticket management, and participant tracking.

In the proposed system, event organizers can create events, update event details, manage participant registrations, and monitor event performance through a dedicated interface. The system also generates **QR code-based tickets** that can be used for quick and secure verification during event entry. This helps reduce the chances of ticket duplication and improves the efficiency of event check-in.

The system also supports **email notifications and event reminders**, ensuring that participants receive important information regarding event schedules and updates. In addition, the platform allows participants to submit feedback after attending an event, which helps organizers analyse participant satisfaction and improve future events.

By integrating all these features into a single system, the proposed solution provides a more efficient, reliable, and user-friendly platform for managing events compared to traditional methods.

2.3.1 Advantages of the Proposed System

The proposed system offers several advantages that improve the efficiency and effectiveness of event management.

- **Provides an online platform** for users to browse and register for events easily.
- **Centralized management system** for organizing and monitoring events.
- **Reduces manual work** involved in registration and record keeping.
- **Improves accuracy** in storing and managing participant information.

- **QR code-based ticket generation** ensures secure event entry verification.
- **Faster event check-in process** during the event.
- **Event organizers can easily create and manage events.**
- **Automated email notifications** keep participants informed about event updates.
- **Better communication** between event organizers and participants.
- **Secure storage of event and participant data.**
- **Feedback collection system** helps evaluate participant experience.
- **Improved monitoring and reporting** of event participation.
- **User-friendly interface** that is easy to access and use.
- **Time-saving system** that simplifies event management tasks.
- **Scalable system** that can support multiple events and large numbers of participants.

2.4 Feasibility Study

Feasibility study is an important step in system analysis that helps determine whether the proposed system is practical and beneficial for implementation. It evaluates different aspects of the system such as technical requirements, operational capability, and economic considerations before development begins. The main objective of a feasibility study is to ensure that the proposed system can be successfully developed and implemented with the available resources.

For the **LiveEvent – Event Management System**, the feasibility study focuses on analysing whether the system can be implemented using available technologies, resources, and infrastructure. The study examines the technical requirements needed to develop the system, the operational benefits it provides to users and organizers, and the overall cost involved in developing and maintaining the system.

The proposed system uses widely available technologies such as **Python, Django, HTML, CSS, and JavaScript**, which makes it technically feasible to develop and

deploy. The system is also designed to be user-friendly so that event organizers and participants can easily use it without requiring advanced technical knowledge. From an operational perspective, the system improves efficiency by automating event registration, ticket generation, and participant management. Economically, the system reduces manual work and administrative costs associated with traditional event management methods.

Overall, the feasibility study shows that the **LiveEvent-Event Management System is practical, efficient, and beneficial for managing events in a digital environment.** There are various types of feasibility to be determined. They are,

- Technical Feasibility
- Operational Feasibility
- Economic feasibility - Cost - Benefit Analysis

2.4.1 Technical Feasibility

Technical feasibility evaluates whether the proposed system can be developed using the available technology, tools, and technical resources. The **LiveEvent – Event Management System** is developed using modern and widely used technologies such as **Python, Django Framework, HTML, CSS, and JavaScript**. These technologies provide a reliable and scalable environment for building web applications.

The system can be implemented using standard computer systems without requiring specialized hardware or expensive software tools. The Django framework also provides built-in features for database management, security, and user authentication, which simplifies the development process. Since these technologies are well-supported and widely available, the proposed system is technically feasible to develop and deploy.

2.4.2 Operational Feasibility

Operational feasibility refers to how well the proposed system will function within the organization and how easily users can adopt it. The **LiveEvent system** is designed with a user-friendly interface that allows users, event organizers, and administrators to perform their tasks easily.

The system simplifies many tasks such as event registration, ticket management, and event monitoring. Participants can easily browse events and register online, while

organizers can manage events through a dedicated dashboard. The automated features such as QR code ticket generation and email notifications improve operational efficiency and reduce manual work. Therefore, the system is operationally feasible and beneficial for both organizers and participants.

2.4.3 Economic Feasibility

Economic feasibility analyses the cost involved in developing and maintaining the proposed system and compares it with the expected benefits. The **LiveEvent System** is developed using open-source technologies such as Python and Django, which reduces the overall development cost.

Since the system is web-based, it can be deployed using existing computer infrastructure without requiring additional expensive hardware. The automation of tasks such as registration, ticket generation, and participant management also reduces administrative effort and operational costs. In the long term, the system helps save time and resources while improving efficiency. Therefore, the proposed system is economically feasible.

2.5 Software Engineering Paradigm Applied

The project follows the **Agile Process Model**. The meaning of Agile is flexible and adaptive. The Agile process model is a software development approach based on iterative and incremental development. In this approach, the development process is divided into smaller parts called iterations. Instead of completing the entire system at once, the project is developed step by step, allowing improvements and changes to be made during development.

The **LiveEvent – Event Management System** was developed using this Agile approach to ensure flexibility and continuous improvement throughout the development process. The requirements of the system were identified at the beginning, but the development was carried out in multiple iterations where new features were gradually implemented and tested. Each iteration focuses on designing, coding, testing, and improving specific parts of the system.

By dividing the project into smaller stages, the Agile model helps reduce risks and allows developers to quickly respond to user requirements and feedback. Each development cycle produces a working version of the system, which can be tested and

improved in the next iteration. This approach helps ensure that the final system meets user expectations and functions efficiently.

Phases in the Agile Model

1. **Requirements Gathering:**
   In this phase, the system requirements are identified and analysed. The needs of users, event organizers, and administrators are studied to determine the functionalities required in the system. This phase helps define the objectives of the LiveEvent system and evaluate its feasibility.

2. **Design the Requirements:**
   After identifying the requirements, the system design is prepared. This includes designing system architecture, database structure, and diagrams such as ER diagrams and UML diagrams. These designs help visualize how the system will function and how different components interact with each other.

3. **Construction / Iteration:**
   During this phase, the actual development of the system takes place. Developers implement the required features such as user registration, event creation, ticket booking, QR code generation, and feedback management. The system is developed incrementally, starting with basic functionality and gradually adding more features.

4. **Testing:**
   In this phase, the developed modules are tested to ensure that the system functions correctly. Testing helps identify and fix errors or bugs in the system. Different types of testing are performed to ensure that all features work as expected.

5. **Deployment:**
   After successful testing, the system is deployed in the working environment where users can access the application. The LiveEvent system becomes available for event organizers and participants to manage and attend events.

6. **Feedback:**
   Once the system is deployed, feedback is collected from users. This feedback helps identify areas for improvement and allows developers to enhance the system in future iterations.

Chapter 3 – System Design

3.1 Introduction

System design is a process of developing specifications for a candidate system that meet the criteria established in the system analysis. Major steps in design are the preparation of the input forms and output reports in a form applicable to user. The main objective of the system design is to use the package easily by any computer operator. System design is the creative act of invention, developing new inputs, a database, offline files, method, procedure and output for processing business to meet an organization objective. System design builds information gathered during the system analysis.

3.2 Database Design

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and effectively. The general objective is to make information easy and quick to access for the users. The general theme behind a database is to integrate all information. Database design is recognized as a standard of management information system and is available virtually for every computer system. In database design several specific objectives are considered.

- Ease of learning and use
- Controlled redundancy
- Data independence
- More information at low cost
- Accuracy and integrity
- Recovery and failure
- Privacy and security

The most important aspect of building an application is the design of tables or the database schema. The data stored in the tables must be organized in some manner, which is meaningful. The overall objective in the process of table design has been to treat data as an organizational, resource and as an integrated whole. The organization of data in a database aim to achieve three major objectives, which are given below. o Data integration o Data abstraction o Data independence Several degrees of normalization have to be applied during the process of table design. The major aim of the process of normalization is to reduce data redundancy and prevent losing data integrity. Data integrity has to be converted at all levels. Pure normalization can access problem related to storage and retrieval of data.

During the process of normalization, dependence's can be identified which cause serious problems during deletion and updating. Normalizing also hope in simplifying the structure of table. Database design is an important activity in design. The efficiency of the system lies in the efficiency of the database. The database design consists of predetermining number of tables that are to be used and fields that are to be used in each table. The efficiency of the system to retrieve the appropriate data depends on how the fields are specified and the coding mostly depends on format of the database. To group the given data and to give them a logical structure, a database design is necessary. The overall objective in the development of the database is to treat data as an organized resource and is an integrated whole.

In this project we are mainly concentrated into relational database. Relational database stores data in table, which is turn, are composed of rows also known as records, columns also known as fields. The fields in the relational models are.,

PRIMARY KEY:

The key which uniquely identifies the records. They also known notify the not null constraint.

FOREIGN KEY:

The key references the primary key, is the data inserted in the primary key column of the table.

The general theme is to handle information as an integrated whole with a minimum of redundancy and improved performance. A complete database management system separates definition of data from programs. The concept of data independence is one of the key advantages of a database management system. The choice of a particular database system is complicated. In this project a relational data model is used because of many features it offers.

Data directory specifies the major element in the system, and care should be taken while designing, in order to avoid unnecessary duplication of data. Several tables are maintaining the system to store data those are required for processing of various data as well storing intermediate for the final processed results. Different modules to meet their needs access these stored data.

NORMALIZATION

Designing a database is complete task and the normalization theory is a useful aid in the design process. The process of normalization is concerned with transformation of conceptual schema into computer representation form. There will be need for most databases to grow by adding new attributes and new relations. The data will be used in new ways. Tuples will be added and deleted. Information stored may undergo updating also. New association may also be added. In such situations the performance of a database is entirely depend upon its design. A bad database design may lead to certain undesirable things like:

1. Repetition of information
2. Inability to represent certain information
3. Loss of information

To minimize these anomalies, Normalization may be used. If the database is in a normalized form, the data can be growing without, in most cases, forcing the rewriting application programs. This is important because of the excessive and growing cost of maintaining an organization's application programs and its data from the disrupting effects of database growth. As the quality of application programs increases, the cost of maintaining the without normalization will rise to prohibitive levels. A normalized database can also encompass many related activities of an organization thereby minimizing the need for rewriting the applications of programs.

First Normal Form

A relation is in first normal form (1NF) if and all its attributes are based on single domain. The objective of normalizing a table is to remove its repeating groups and ensure that all entries of the resulting table have at most single value.

Second Normal Form

A table is said to be second Normal Form (2NF), when it is in 1NF and every attribute in the record is functionally dependent upon the whole key, and not just a part of the key.

Third Normal Form

A table is in third Normal Form (3NF), when it is in 2NF and every non-key attribute is functionally dependent on just the primary key.

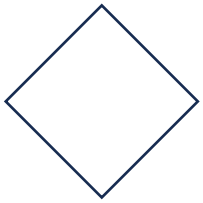
3.2.1 Entity Relationship Model

An entity relationship diagram (ERD) is a data modelling technique that graphically illustrates an information system's entities and the relationships between those entities. An ERD is a conceptual and representational model of data used to represent the entity framework infrastructure. The elements of an ERD are:

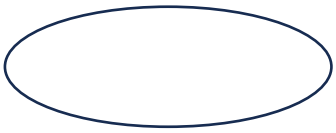
- Entities:



- Relationships:

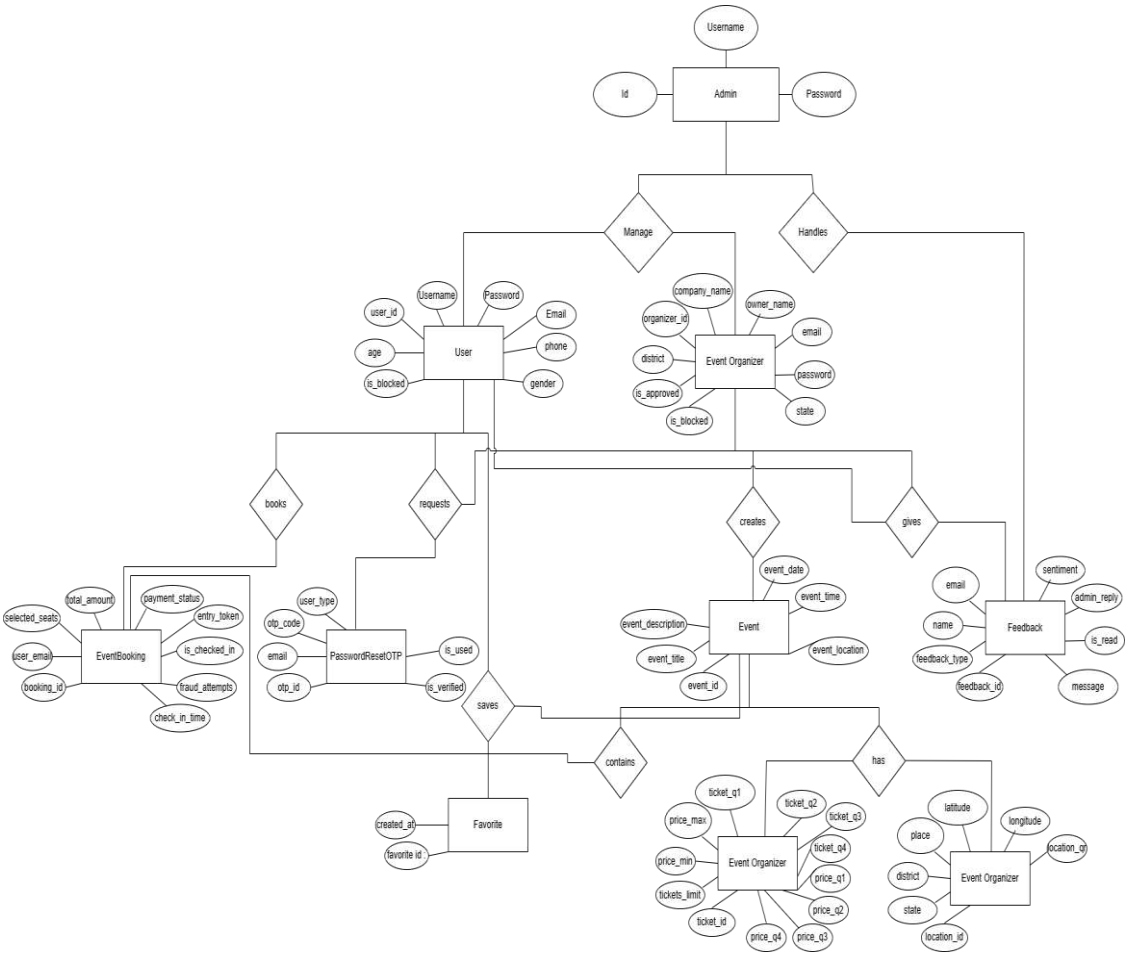


- Attributes:



LiveEvent

ER-DIAGRAM



3.2.2 Data Dictionary

Data dictionary is a collection of information about data elements used in the system. It describes the structure of the database tables, including the field names, data types, and their descriptions. The data dictionary helps developers understand how data is stored and maintained in the database. It also ensures consistency and clarity in database design.

In the **LiveEvent – Event Management System**, several tables are used to store information related to users, event organizers, events, bookings, tickets, feedback, and system operations. Each table contains specific fields that store relevant information required for managing the event platform efficiently.

TABLES

Table No: 1

Table Name: user_register

Description: This table is used to store the details of registered users.

Primary Key: id

Table 3.1: user_register

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key and auto increment column
user_name	varchar(100)	Used to store the name of the user
photo	varchar(200)	Used to store the photo of the user
age	int	Used to store the age of the user
gender	varchar(10)	Used to store the gender of the user
email	varchar(100)	Used to store email address
phone	varchar(10)	Used to store phone number
password	varchar(128)	Used to store user password
is_blocked	boolean	Used to indicate whether user is blocked

Table No: 2

Table Name: EventRegister

Description: This table is used to store event organizer details.

Primary Key: id

Table 3.2: EventRegister

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key and auto increment column
main_category	varchar(50)	Used to store main category of event
event_type	varchar(100)	Used to store event type
company_name	varchar(100)	Used to store company name
owner_name	varchar(100)	Used to store organizer name
profile_image	varchar(200)	Used to store organizer profile image
id_proof	varchar(200)	Used to store organizer ID proof
website	varchar(200)	Used to store website link
state	varchar(50)	Used to store state
district	varchar(50)	Used to store district
address	text	Used to store address
email	varchar(100)	Used to store organizer email
password	varchar(128)	Used to store password
created_at	datetime	Used to store registration time
is_approved	boolean	Used to store approval status
is_blocked	boolean	Used to store blocked status

Table No: 3

Table Name: EventCreate

Description: This table stores event details created by organizers.

Primary Key: event_id

Foreign Key: created_by

Table 3.3: EventCreate

FIELD NAME	DATA TYPE	DESCRIPTION
event_id	bigint	Used as primary key
event_title	varchar(200)	Used to store event title
instagram_url	varchar(200)	Used to store Instagram link
event_description	text	Used to store event description
event_date	date	Used to store event date
event_time	time	Used to store event time
event_location	varchar(200)	Used to store event location
event_image	varchar(200)	Used to store event image
created_by	bigint	Used to store organizer reference
created_at	datetime	Used to store event creation time
updated_at	datetime	Used to store event update time

Table No: 4

Table Name: Favorite

Description: This table stores user favorite events.

Primary Key: id

Foreign Key: user, event

Table 3.4: Favorite

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
user	bigint	Used to store user reference
event	bigint	Used to store event reference
created_at	datetime	Used to store favorite creation time

Table No: 5

Table Name: EventLocation

Description: This table stores event location details.

Primary Key: id

Foreign Key: event

Table 3.5: EventLocation

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
event	bigint	Used to store event reference
state	varchar(50)	Used to store state
district	varchar(50)	Used to store district
place	varchar(100)	Used to store place
address	text	Used to store address
latitude	decimal	Used to store latitude value
longitude	decimal	Used to store longitude value
location_qr	varchar(200)	Used to store location QR code

Table No: 6

Table Name: EventContact

Description: This table stores contact information of events.

Primary Key: id

Foreign Key: event

Table 3.6: EventContact

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
event	bigint	Used to store event reference
contact_email	varchar(100)	Used to store contact email
contact_phone	varchar(15)	Used to store contact phone number

Table No: 7

Table Name: EventTicket

Description: This table stores ticket and pricing details for events.

Primary Key: id

Foreign Key: event

Table 3.7: EventTicket

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
event	bigint	Used to store event reference
payment_mode	varchar(50)	Used to store payment mode
tickets_limit	int	Used to store ticket limit
price_min	decimal	Used to store minimum price
price_max	decimal	Used to store maximum price
ticket_q1	int	Used to store quantity of tier 1 tickets

LiveEvent

ticket_q2	int	Used to store quantity of tier 2 tickets
ticket_q3	int	Used to store quantity of tier 3 tickets
ticket_q4	int	Used to store quantity of tier 4 tickets
price_q1	decimal	Used to store price of tier 1 ticket
price_q2	decimal	Used to store price of tier 2 ticket
price_q3	decimal	Used to store price of tier 3 ticket
price_q4	decimal	Used to store price of tier 4 ticket

Table No: 8

Table Name: EventBooking

Description: This table stores booking details for events.

Primary Key: id

Foreign Key: event

Table 3.8: EventBooking

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
event	bigint	Used to store event reference
user_email	varchar(100)	Used to store user email
selected_seats	text	Used to store selected seats
total_amount	decimal	Used to store payment amount
payment_status	varchar(20)	Used to store payment status

LiveEvent

payment_id	varchar(100)	Used to store payment transaction ID
check_in_time	datetime	Used to store event check-in time
created_at	datetime	Used to store booking time
entry_token	varchar(100)	Used for entry verification
is_checked_in	boolean	Used to store check-in status
fraud_attempts	int	Used to store fraud attempt count
last_scan_attempt	datetime	Used to store last scan attempt
scan_ip	varchar(50)	Used to store scan IP address
email_status	varchar(20)	Used to store email sending status
email_attempts	int	Used to store email attempt count

Table No: 9

Table Name: Feedback

Description: This table stores feedback details from users and organizers.

Primary Key: id

Table 3.9: Feedback

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
feedback_type	varchar(20)	Used to store feedback type
name	varchar(100)	Used to store user name
email	varchar(100)	Used to store email

LiveEvent

message	text	Used to store feedback message
screenshot	varchar(200)	Used to store feedback screenshot
sentiment	varchar(20)	Used to store sentiment analysis result
is_read	boolean	Used to track if feedback is read
admin_reply	text	Used to store admin reply
replied_at	datetime	Used to store reply time
created_at	datetime	Used to store feedback time

Table No: 10

Table Name: PasswordResetOTP

Description: This table stores OTP details for password reset functionality.

Primary Key: id

Table 3.10: PasswordResetOTP

FIELD NAME	DATA TYPE	DESCRIPTION
id	bigint	Used as primary key
email	varchar(100)	Used to store user email
otp_code	varchar(6)	Used to store OTP code
user_type	varchar(20)	Used to store type of user
created_at	datetime	Used to store OTP creation time
attempt_count	int	Used to store OTP attempt count
is_verified	boolean	Used to check verification status
is_used	boolean	Used to check if OTP is used

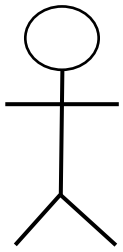
3.3 Object Oriented Design-Uml Diagram

3.3.1 Usecase Diagram

The most important aspect is to capture the dynamic behavior when modelling a system. Dynamic means the behavior of the system when it is operating. Use case diagram is one of the diagrams that describe the dynamic behavior of the system. Use case diagrams are used to gather the requirements of a system including internal and external influences. These requirements are mostly design requirements. Hence, when a system analyzed to gather its functionalities, use cases are prepared and actors are identified. When the initial task is complete, use case diagrams are modelled to present the outside view.

Symbols in Use-case Diagram:

An actor represents a coherent set of roles that users of a system play when interacting with the use cases of the system. Actors can be anything-humans, devices, other systems. One physical object can play several roles and so can be modelled by several actors.



Use cases describe what a system does, not how it does it. A use case contains multiple scenarios, each of which describes a specific flow of events through the use case. Use case behavior is specified by describing the scenarios clearly enough for outsiders to understand.

Connection symbol represents the connection between actor and use cases**.**

Include symbol include the relationship between use cases. One use case(base) includes the functionality of another (inclusion case).

LiveEvent

USECASE DIAGRAM

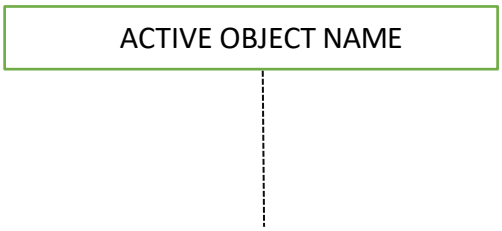


3.3.2 Sequence Diagram

A sequence diagram is an interaction diagram that shows how objects operate with one another and in what order. It is a constant of a message sequence chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the logical view of the system under development

3.3.2.1 Symbols in Sequence Diagram

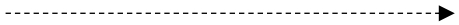
- **Life lines & life line boxes: -** Active objects can be an object which is an instance of a class and can be drawn with rectangular box called the life line box with its name is specified within the box. The long dashed line tailing the object is called life lines.



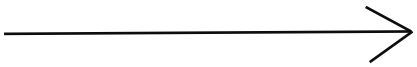
- **Message:** -It is used to illustrate the communication between different active objects. Each message between object is represented with a message, expressions and filled arrow solid lines from the calling active objects life line to the recipient life lines.

- **Synchronous:** - This type of message is used when it is important that a message is received & completed before execution of control flow begins.

- **Return**: -Return message shows that the control flow has return into the calling active objects and the synchronous messages are completed its operations.



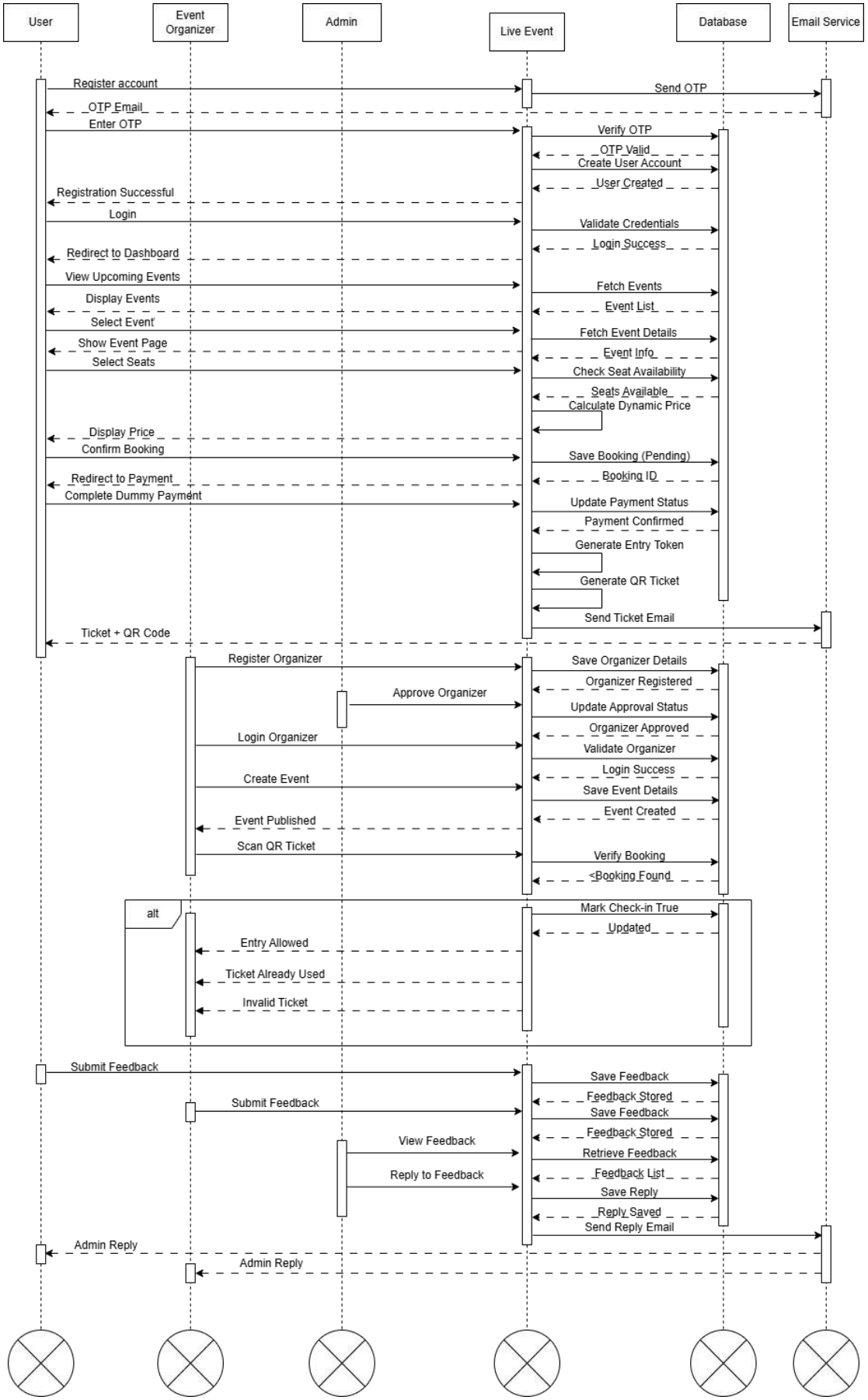
- **Asynchronous**: - This type of message is sent from an active object and waits for a responds. This type of message is used when control flow doesn't lead to interrupted before complete the operation.



- **Flat:** -This type of message is used when there is no distinction between asynchronous and synchronous messages

LiveEvent

SEQUENCE DIAGRAM

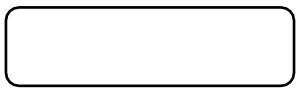


3.3.3 Activity Diagram

Activity diagrams are graphical representation of workflows of stepwise activities and actions with support for choice, iteration and concurrency. UML, activity diagrams are intended to model both computational and organizational process. It shows the overall flow of control.

Symbols in Activity Diagram

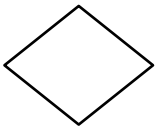
Action State: - It is an automatic state once it started execution. It will come to completion without any interruption at the middle of execution.



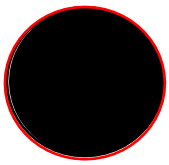
Transition: - When the action or activity state completes the flow of control passes immediately to the next action or activity state. This is represented by using the solid lines.



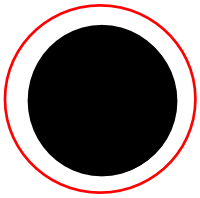
Branching: - It specifies the alternative paths that are taken based on some Boolean expressions.



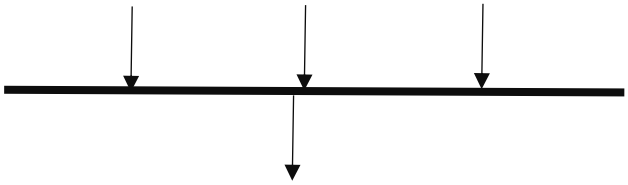
Initial node: - The initial node is a control node from where the activity is invoked. An activity may have more than one initial node.



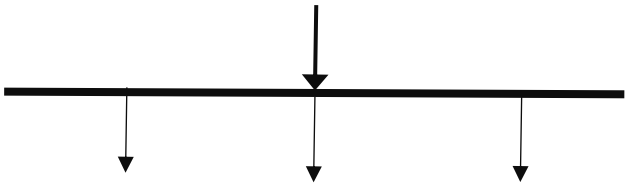
Final node: - The final node shows the end of the activity. An activity can have more than one final node but the first one reached will stop the activity.



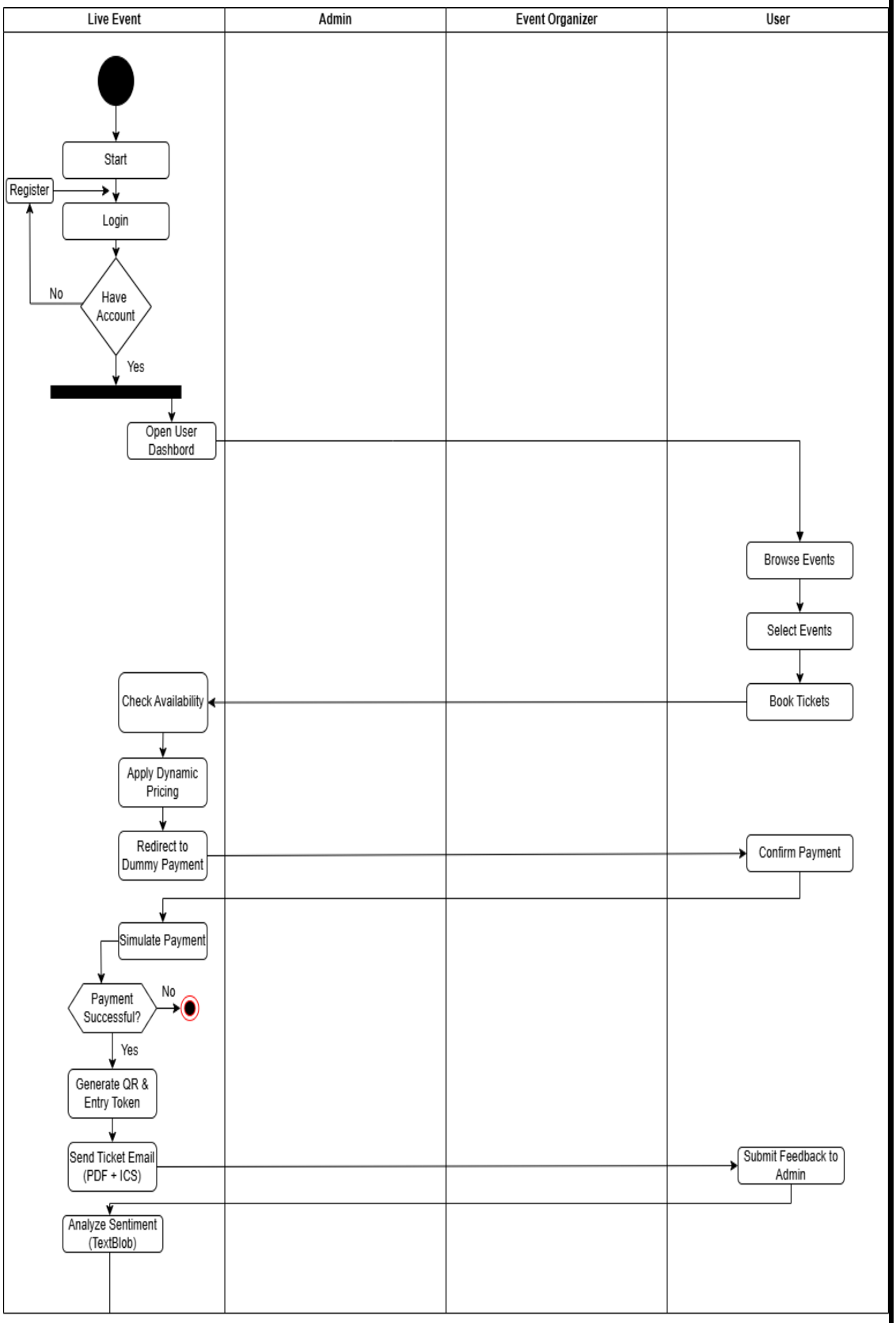
Join: -A join represents multiple incoming transactions and provide only one outgoing transitions.



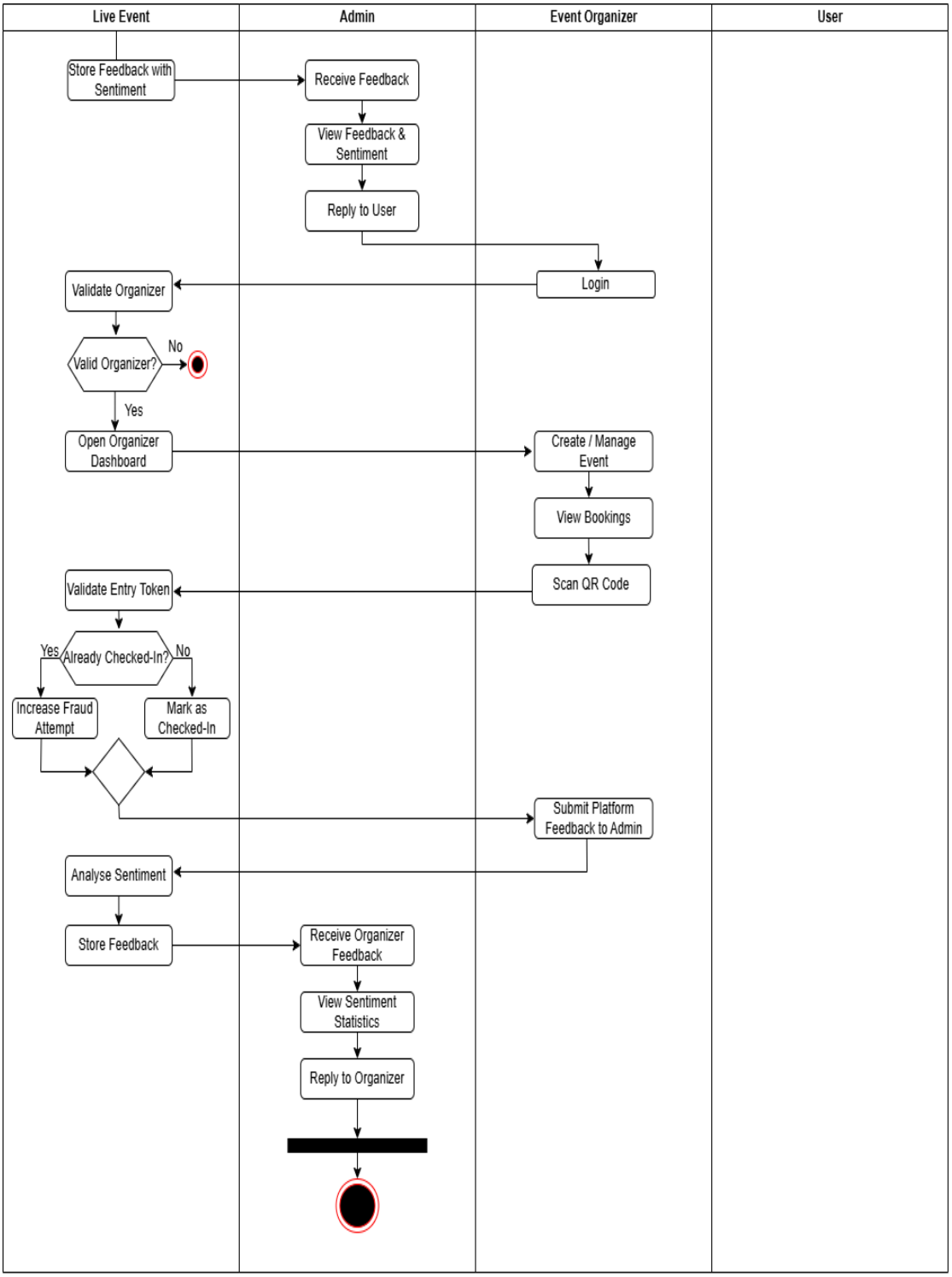
Fork: -A fork represents only one incoming transition and provide multiple outgoing transitions.



ACTIVITY DIAGRAM



LiveEvent


Live Event
Admin
Event Organizer
User
Store Feedback with Sentiment
Receive Feedback
View Feedback & Sentiment
Reply to User
Login
Validate Organizer
Valid Organizer?
No
Yes
Open Organizer Dashboard
Create / Manage Event
View Bookings
Scan QR Code
Validate Entry Token
Already Checked-In?
Yes
No
Increase Fraud Attempt
Mark as Checked-In
Submit Platform Feedback to Admin
Analyse Sentiment
Store Feedback
Receive Organizer Feedback
View Sentiment Statistics
Reply to Organizer

3.4 Modular Design

Modular design is a software architecture approach that involves dividing a system into smaller, independent components known as modules. Each module is responsible for performing a specific set of functions and can be developed, tested, and maintained separately from other modules. This approach improves the overall structure of the software system and makes development more organized and efficient.

In the **LiveEvent – Event Management System**, the application is divided into several modules such as the **User Module, Event Organizer Module, Admin Module, Event Management Module, Ticket Management Module, and Feedback Module**. Each module performs specific tasks related to event management and user interaction. By dividing the system into modules, it becomes easier to manage complex functionalities and maintain the system effectively.

The modular approach also allows developers to improve or update individual modules without affecting the entire system. This ensures better scalability, maintainability, and flexibility in the development process.

Key characteristics of a modular design in software engineering include:

• **Encapsulation:** Each module hides its internal implementation details and exposes only the necessary interface for communication with other modules. This ensures that changes within one module do not affect other modules in the system.

• **Abstraction:** Modules provide clear and simple interfaces that allow other modules to use their functionalities without understanding the internal complexity. This helps reduce system complexity and improves system readability.

• **Independence:** Modules are designed as independent units that can be developed and tested separately. This makes it easier to identify errors and allows multiple developers to work on different modules simultaneously.

• **Reusability:** Modules developed for one system can often be reused in other projects. This reduces development time and improves efficiency in software development.

• **Maintainability:** Since each module performs a specific function, updating or modifying the system becomes easier. Changes in one module do not affect other modules, making maintenance simpler.

• **Flexibility:** The modular structure allows the system to be easily extended by adding new modules or improving existing ones without modifying the entire system.

The process of modularization involves identifying different functionalities of the system, defining the responsibilities of each module, and establishing communication between modules through well-defined interfaces.

3.4.1 Modules Description

1. USER MODULE

a. User Registration

- New users can register in the system by providing details such as name, email, phone number, password, and profile information.
- The system validates the information and creates a user account.
- Registered users can then access the system and participate in events.

b. User Login

- Registered users can log in using their email and password.
- Authentication ensures secure access to the system.
- Once logged in, users can access available events and manage their bookings.

c. Browse Events

- Users can view a list of upcoming events.
- Each event displays details such as event title, description, date, time, and location.
- Users can explore events and select the ones they are interested in attending.

d. Ticket Booking

- Users can book tickets for available events.
- The system allows users to select ticket quantities or seats based on event availability.
- Booking information is stored in the database for future reference.

e. Ticket Download

- After successful booking, users can download their digital tickets.
- Tickets contain event details and a **QR code** for entry verification during the event.

f. favourite

- Users can mark events as favourites.
- Favourite events can be easily accessed later without searching again.

g. Feedback

- Users can submit feedback about events or the platform.
- Feedback helps organizers and administrators improve event quality and user experience.

2. EVENT ORGANIZER MODULE

a. Organizer Registration

- Event organizers can register in the system by providing details such as company name, owner name, email, contact information, and address.
- The system verifies the organizer details and stores the registration information in the database.
- After registration, the organizer account may require admin approval before accessing all system features.

b. Organizer Login

- Registered organizers can log in to the system using their email and password.
- The login process ensures secure authentication for event organizers.
- After logging in, organizers can access the dashboard to manage their events.

c. Event Creation

- Organizers can create new events by providing details such as event title, description, date, time, and location.

• They can also upload event images and provide social media links if needed.

• Once created, the event becomes available for users to browse and register.

d. Event Management

• Organizers can update, edit, or delete existing events.

• They can manage event details such as schedule, venue, and description.

• This feature allows organizers to keep event information accurate and up to date.

e. Ticket Configuration

• Organizers can configure ticket details for each event.

• They can define ticket limits, pricing ranges, and ticket tiers.

• This feature allows flexible ticket management for different types of events.

f. Event Analytics

• Organizers can view analytics related to their events.

• This includes information such as number of bookings, ticket sales, and participant details.

• Analytics help organizers evaluate the success of their events.

g. Attendance Scanner

• Organizers can verify participant entry using QR code scanning.

• The scanner checks the authenticity of the ticket and marks attendance.

• This helps prevent fraud and ensures a smooth check-in process during the event.

3. ADMIN MODULE

a. Admin Dashboard

• The admin dashboard provides an overview of the entire system.

• It displays important information such as total users, organizers, events, and bookings.

• The dashboard helps the administrator monitor system activities and performance.

b. Manage Users

- The admin can view, edit, or remove user accounts from the system.
- Admin has the authority to block or unblock users if necessary.
- This feature helps maintain proper control over user access and system usage.

c. Manage Organizers

- The admin can review organizer registration requests.
- Admin can approve or reject organizer accounts based on verification.
- Organizers can also be blocked or managed if they violate system policies.

d. Manage Events

- The admin can view and monitor all events created by organizers.
- Admin can edit or remove events if they do not meet system guidelines.
- This ensures that only valid and appropriate events are available to users.

e. Booking Management

- The admin can view all event booking records in the system.
- Booking details such as user email, event information, and payment status can be monitored.
- This helps the administrator manage event participation and detect suspicious activities.

f. Feedback Management

- The admin can view feedback submitted by users and organizers.
- Feedback messages can be analyzed and marked as read.
- The admin can also reply to feedback to address issues or suggestions from users.

4. TICKET MANAGEMENT MODULE

a. Dynamic Pricing

- The system allows organizers to configure different ticket prices for events.
- Ticket prices can vary based on categories, availability, or special offers.

• This feature helps organizers manage ticket sales effectively and maximize participation.

b. Seat Selection

• Users can select available seats or ticket quantities while booking an event.

• The system ensures that seat availability is updated automatically after each booking.

• This helps avoid double booking and ensures accurate ticket allocation.

c. QR Code Ticket Generation

• After successful ticket booking, the system automatically generates a unique QR code for each ticket.

• The QR code contains essential booking information used for event entry verification.

• This feature helps ensure secure and quick check-in during the event.

d. Email Ticket Delivery

• Once the booking is confirmed, the ticket details are sent to the user through email.

• The email includes event details and the QR code ticket for entry.

• This ensures that users can easily access their tickets anytime.

5. FEEDBACK AND SENTIMENT ANALYSIS MODULE

a. Feedback Submission

• Users and event organizers can submit feedback about events or the platform.

• The feedback form allows users to provide their name, email, and message describing their experience.

• Users can also attach screenshots if they want to report issues or provide additional information.

b. Sentiment Analysis

• The system analyzes the feedback message using sentiment analysis techniques.

• Based on the content of the message, the system categorizes the feedback as **positive, negative, or neutral**.

• This helps administrators quickly understand user opinions and identify areas that require improvement.

c. Admin Reply

• The administrator can review feedback submitted by users and organizers.

• Admin can respond to feedback messages to resolve issues or provide clarification.

• This feature improves communication between the system administrators and users.

3.5 Input Design

The user interface design is very important for any application. It describes how the software communicates with users and other system components. Input design is the process of converting user-oriented data into a computer-readable format.

In the **LiveEvent System**, data is entered through simple and interactive web forms such as user registration, login, event creation, ticket booking, and feedback submission. These forms are designed to be user-friendly so that users can easily enter the required information.

Proper validation is provided to ensure that only correct and complete data is stored in the system. Error messages and guidance are also included to help users enter data correctly. The main goal of input design is to make the system easy to use and minimize errors during data entry.

The basic steps involved in input design are:

•Review input requirements

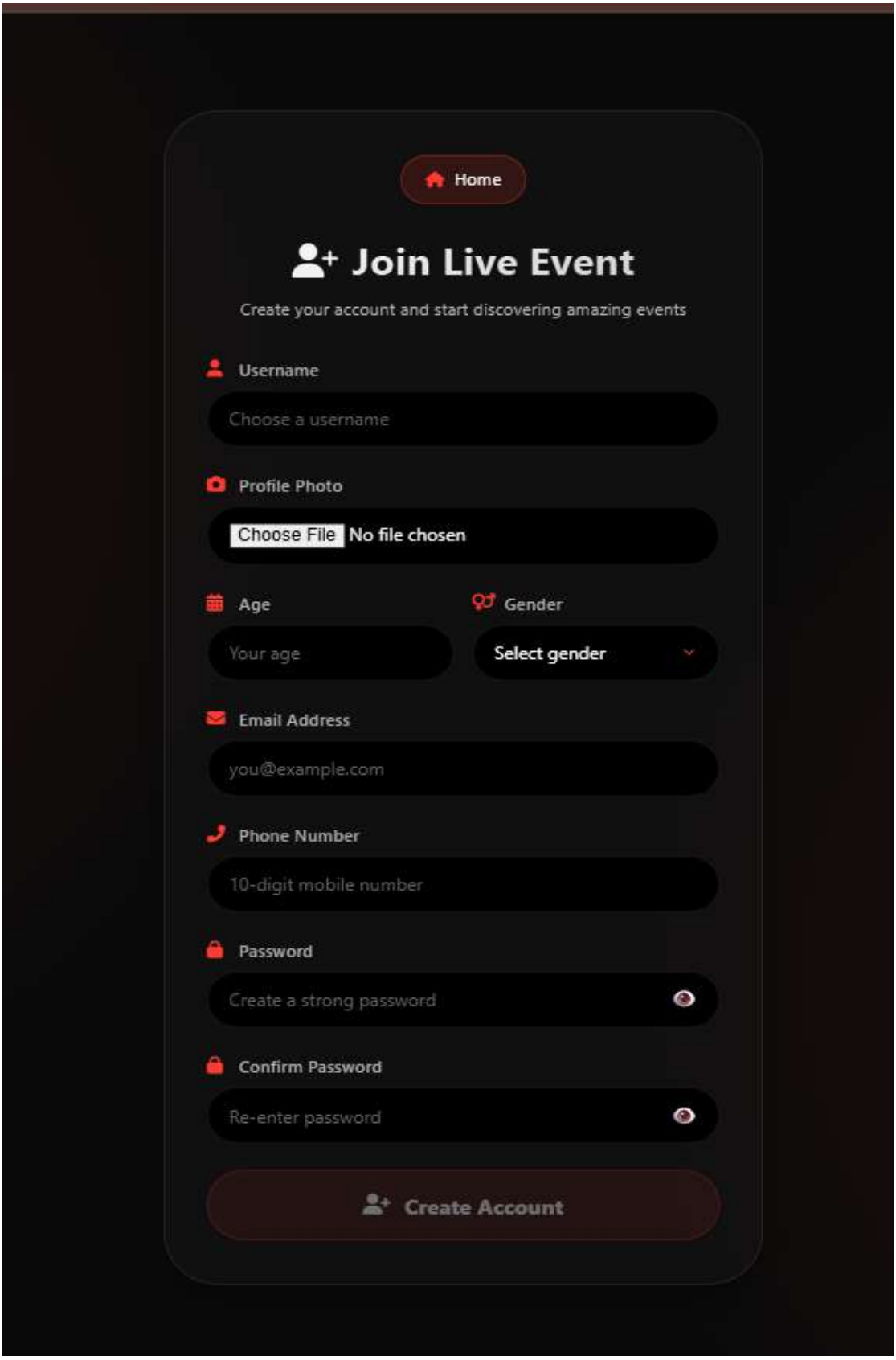
• Decide how the input data flow will be implemented

• Identify the source of input data

• Design the input screens with proper validation

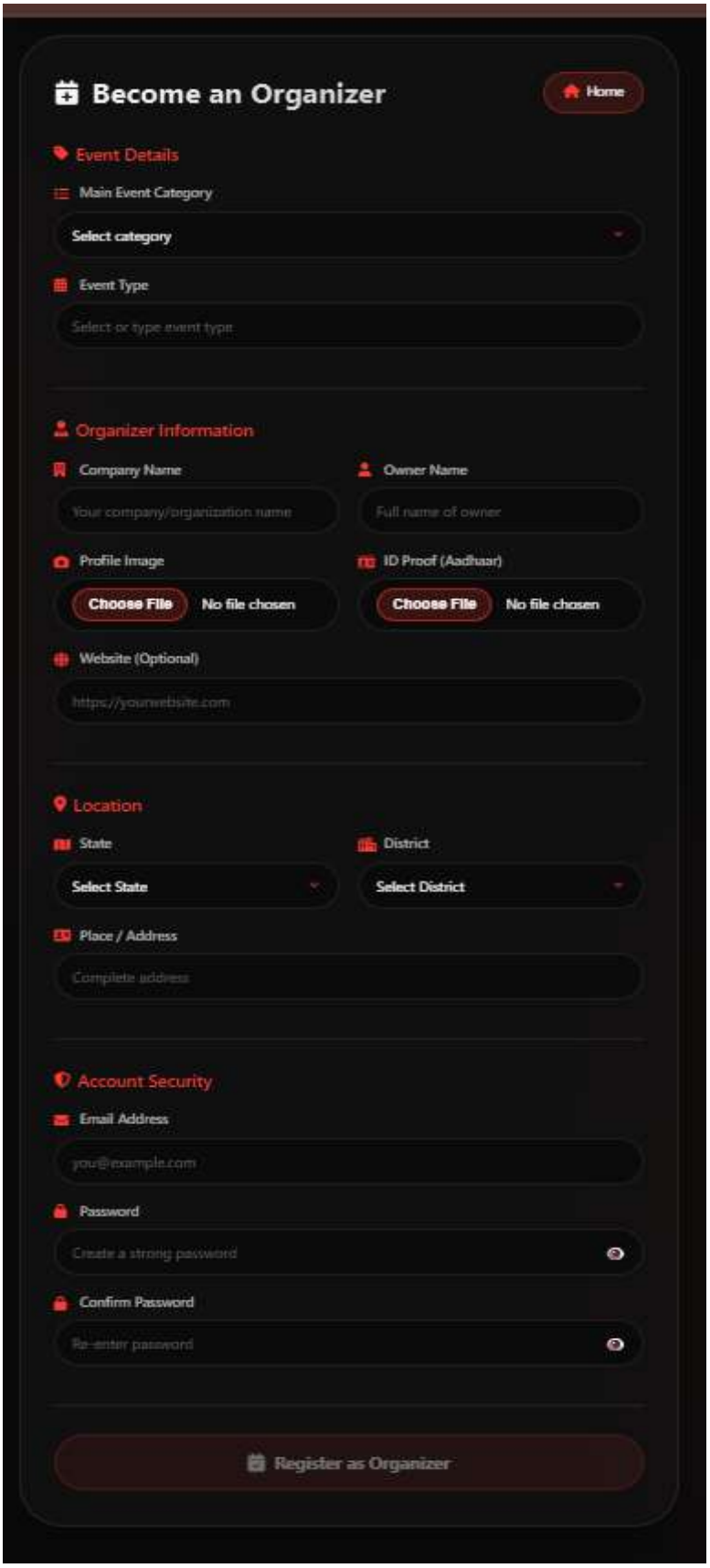
SCREENSHOTS

User Registration Form

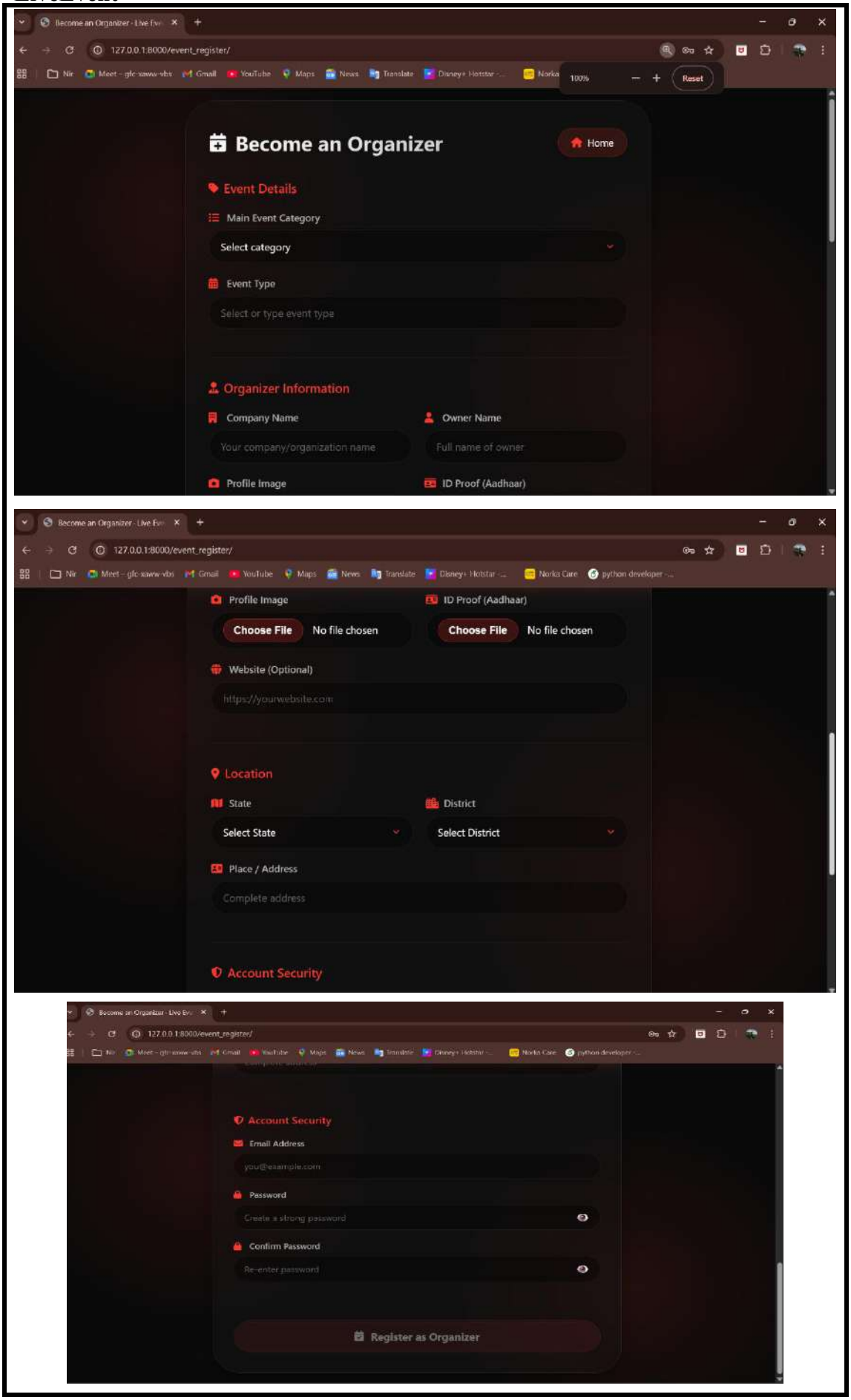


LiveEvent

Organizer Registration Form

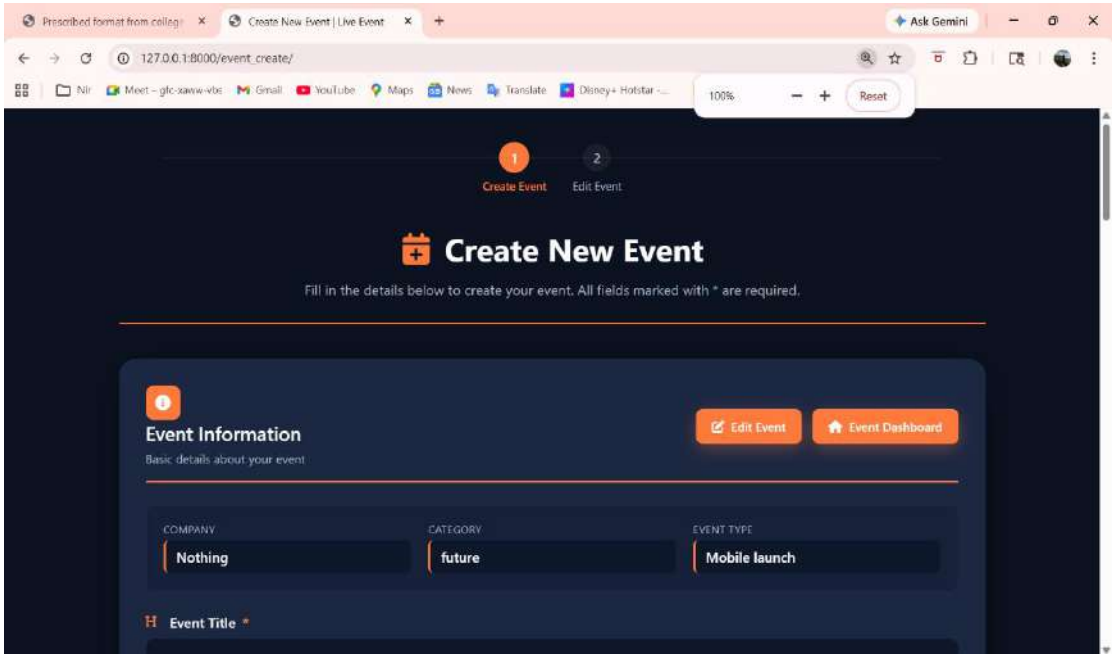


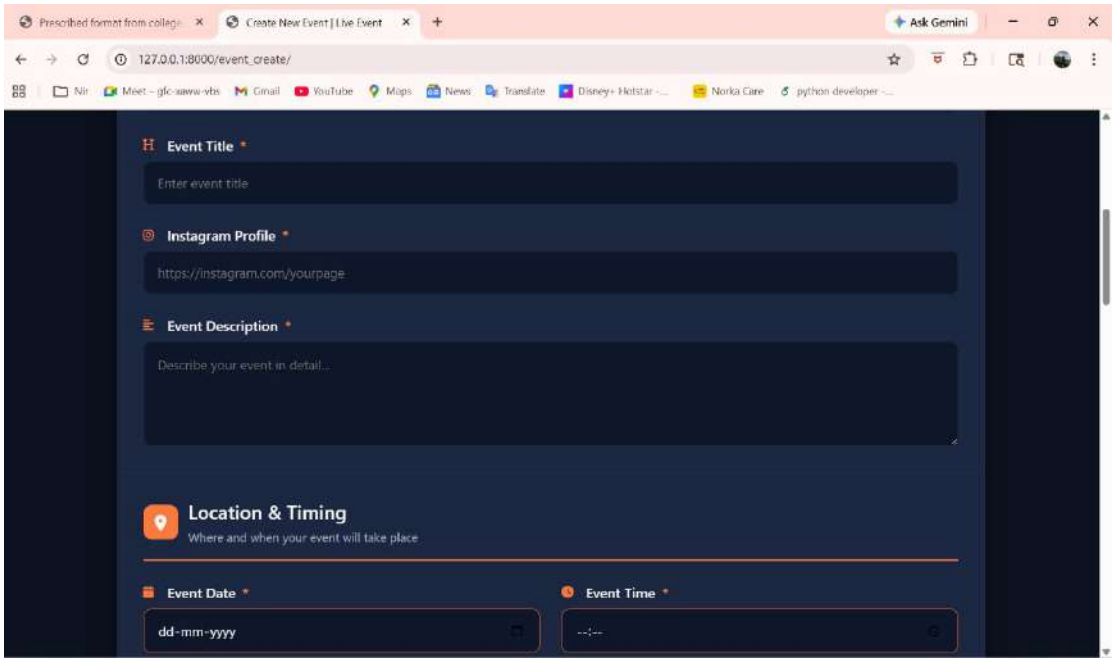
LiveEvent


127.0.0.1:8000/event_register/
Become an Organizer
Home
Event Details
Main Event Category
Select category
Event Type
Select or type event type
Organizer Information
Company Name
Your company/organization name
Owner Name
Full name of owner
Profile Image
ID Proof (Aadhaar)
Choose File
No file chosen
Website (Optional)
https://yourwebsite.com
Location
State
Select State
District
Select District
Place / Address
Complete address
Account Security
Email Address
you@example.com
Password
Create a strong password
Confirm Password
Re-enter password
Register as Organizer

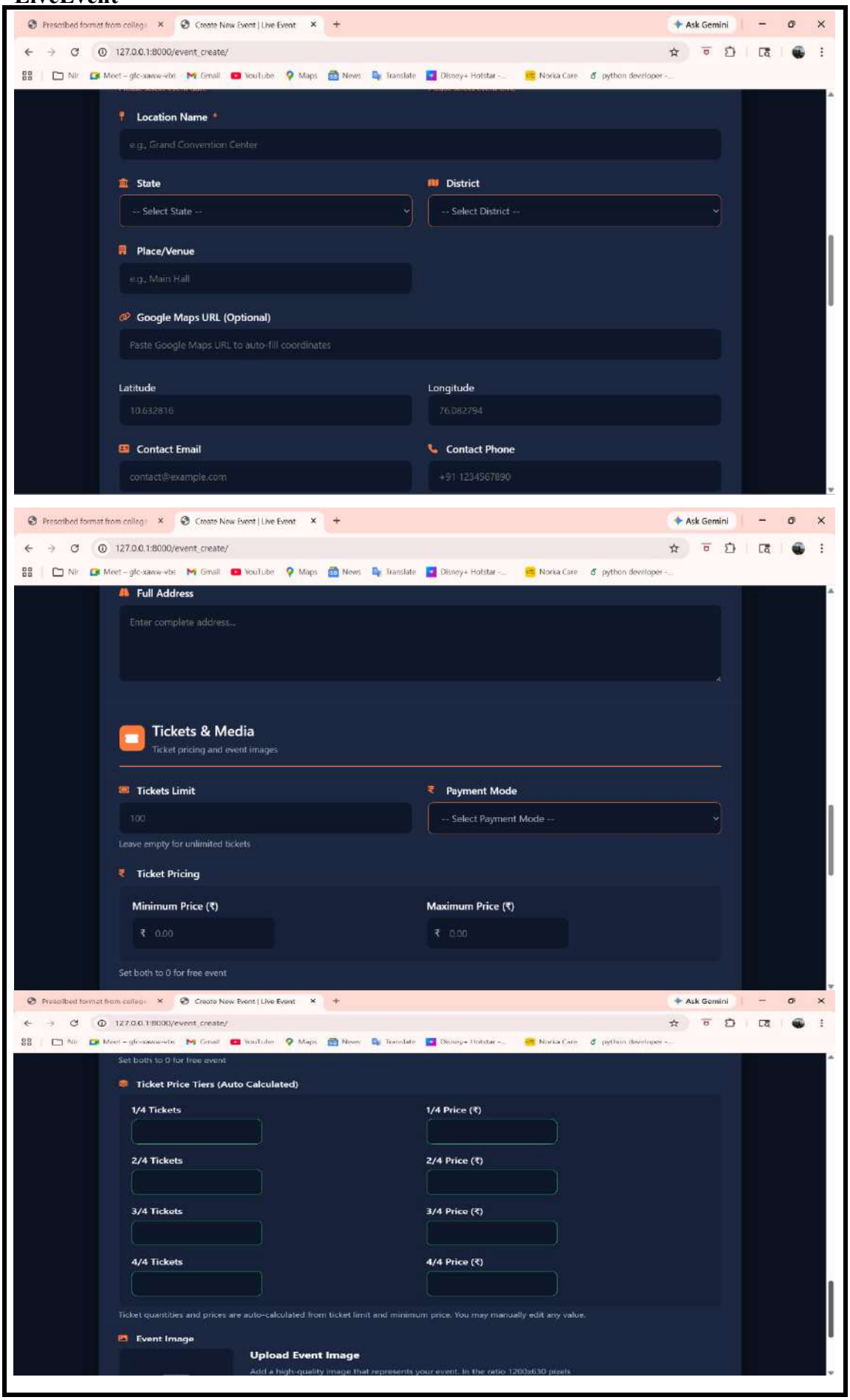
LiveEvent

□ Event Creation Form

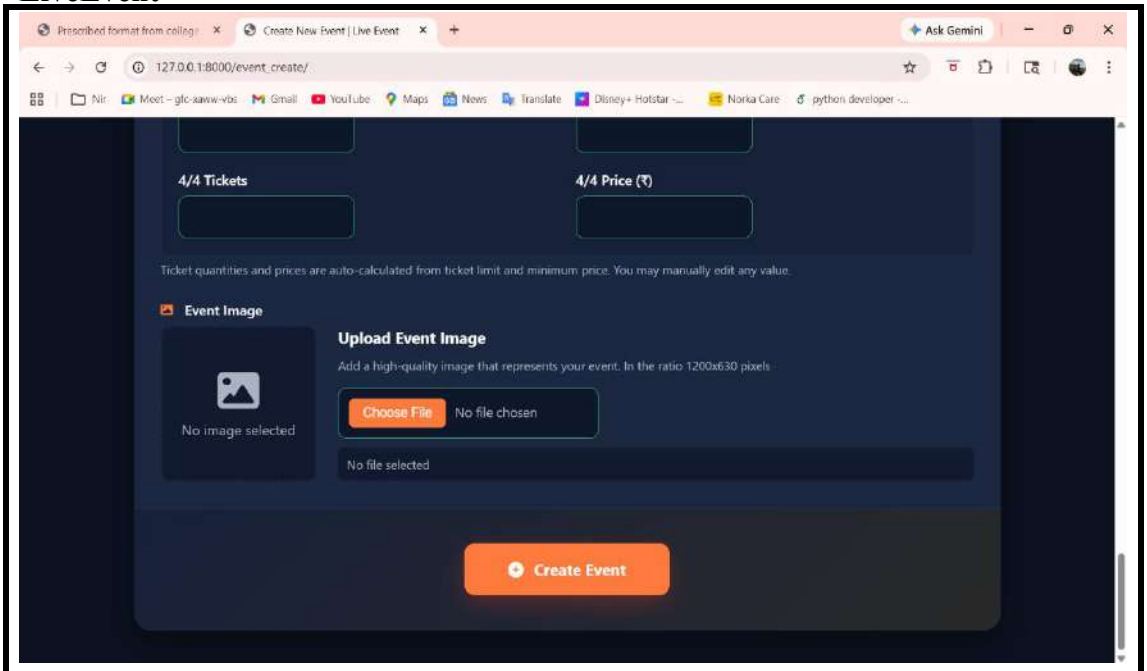




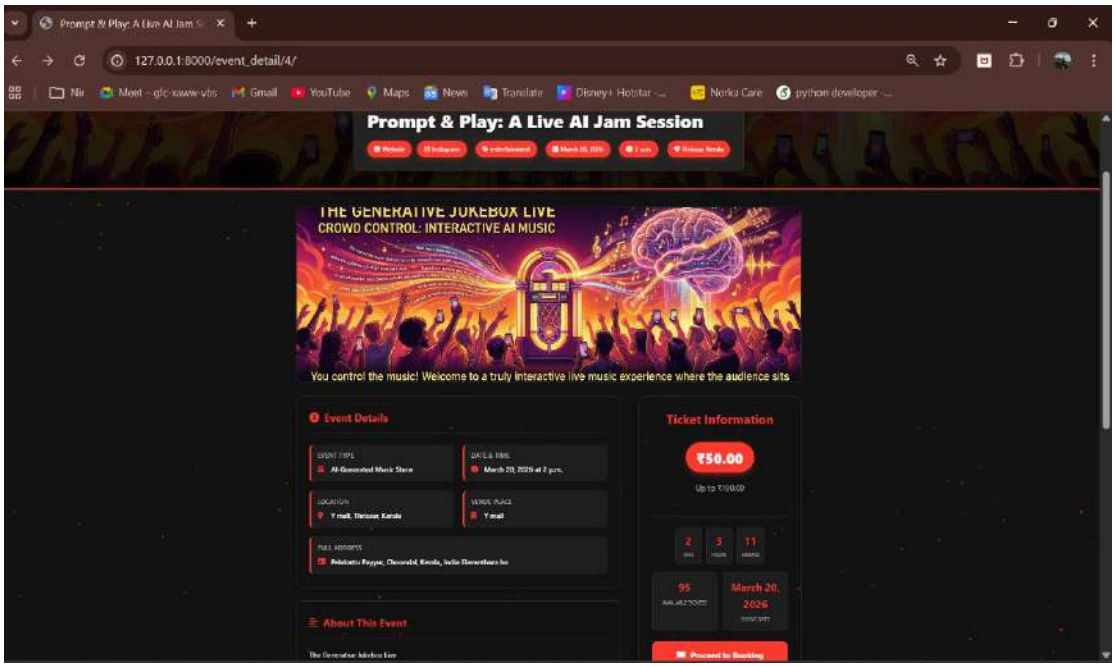
LiveEvent


Create New Event | Live Event
127.0.0.1:8000/event_create/
Ask Gemini
Location Name *
e.g., Grand Convention Center
State
-- Select State --
District
-- Select District --
Place/Venue
e.g., Main Hall
Google Maps URL (Optional)
Paste Google Maps URL to auto-fill coordinates
Latitude
10.632816
Longitude
76.082794
Contact Email
contact@example.com
Contact Phone
+91 1234567890
Full Address
Enter complete address...
Tickets & Media
Ticket pricing and event images
Tickets Limit
100
Leave empty for unlimited tickets
Payment Mode
-- Select Payment Mode --
Ticket Pricing
Minimum Price (₹)
₹ 0.00
Maximum Price (₹)
₹ 0.00
Set both to 0 for free event
Ticket Price Tiers (Auto Calculated)
1/4 Tickets
1/4 Price (₹)
2/4 Tickets
2/4 Price (₹)
3/4 Tickets
3/4 Price (₹)
4/4 Tickets
4/4 Price (₹)
Ticket quantities and prices are auto-calculated from ticket limit and minimum price. You may manually edit any value.
Event Image
Upload Event Image

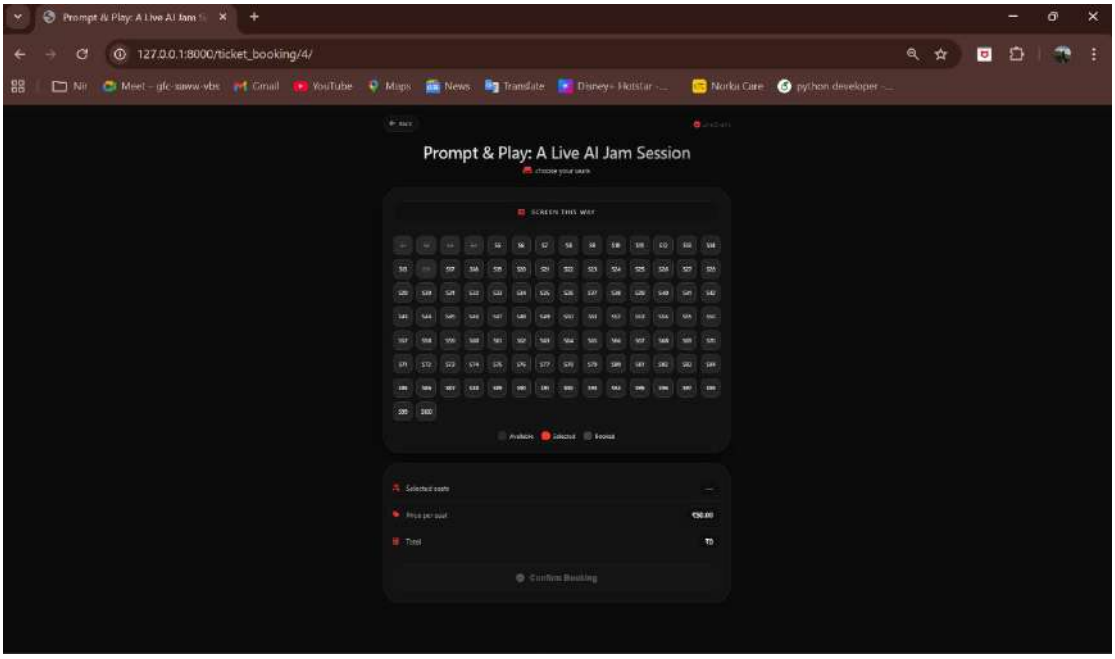
LiveEvent

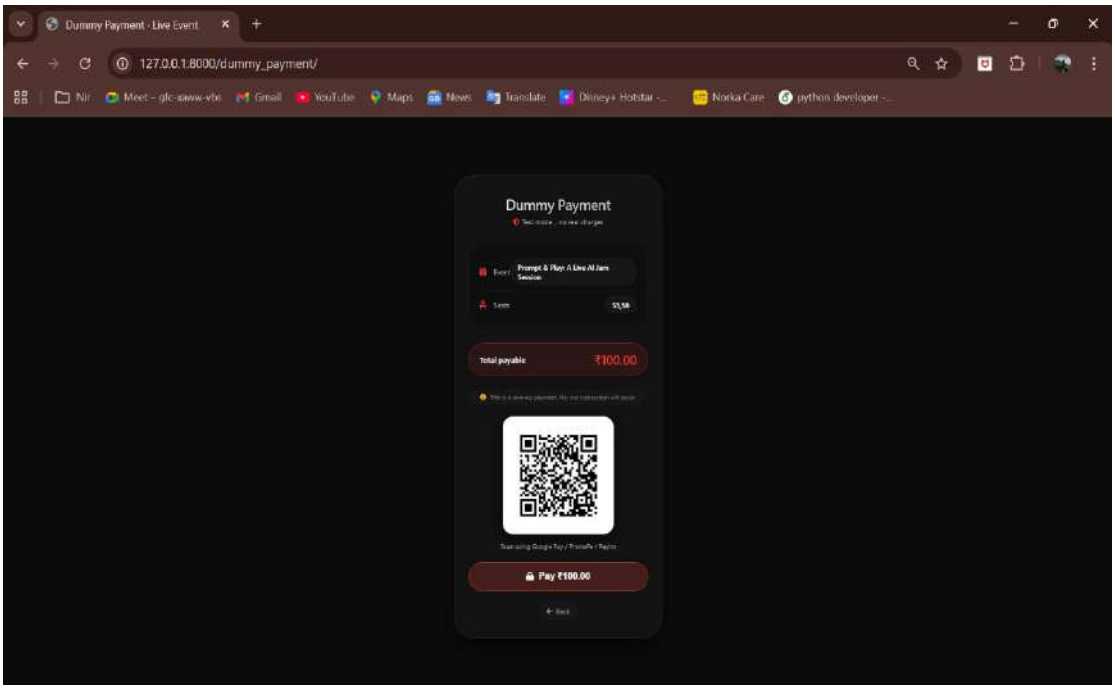


Ticket Booking Form



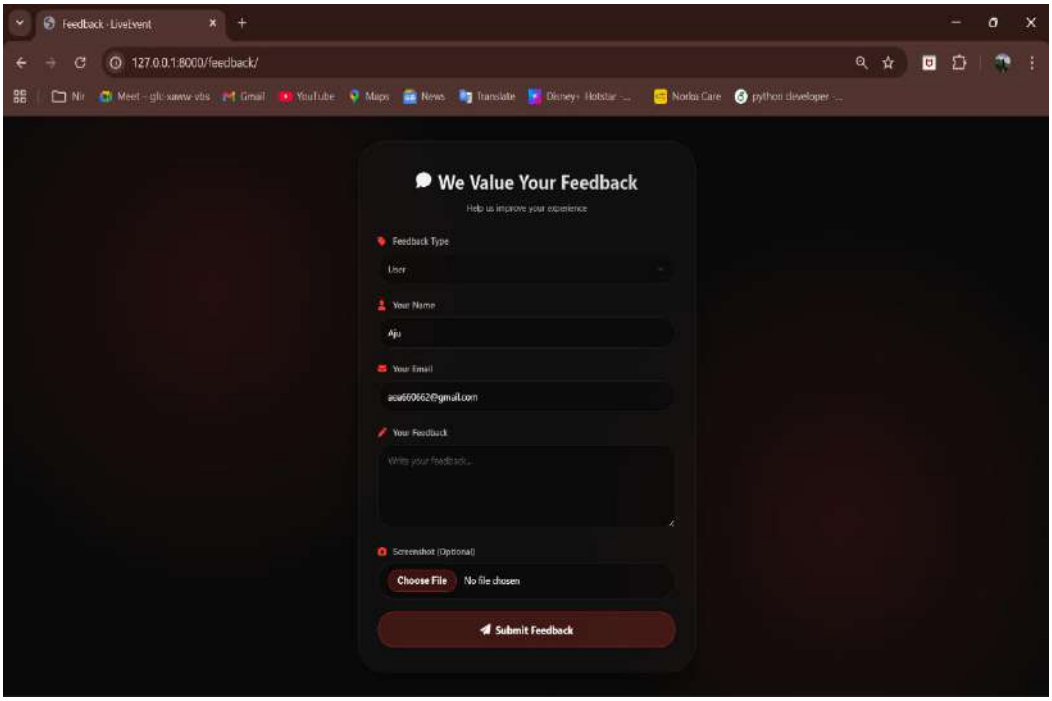
LiveEvent


127.0.0.1:8000/ticket_booking/4/
Prompt & Play: A Live AI Jam Session


Dummy Payment · Live Event
127.0.0.1:8000/dummy_payment/
Dummy Payment
Total payable
₹100.00
Pay ₹100.00

LiveEvent

Feedback Form



3.6 Output Design

Output design is the process of presenting processed information to the users in a clear and understandable format. The main objective of output design is to provide useful information to users so that they can easily understand the results generated by the system.

In the **LiveEvent System**, outputs are displayed through web pages and generated reports. The system provides outputs such as event listings, booking confirmations, QR code tickets, feedback results, and event reports for organizers and administrators.

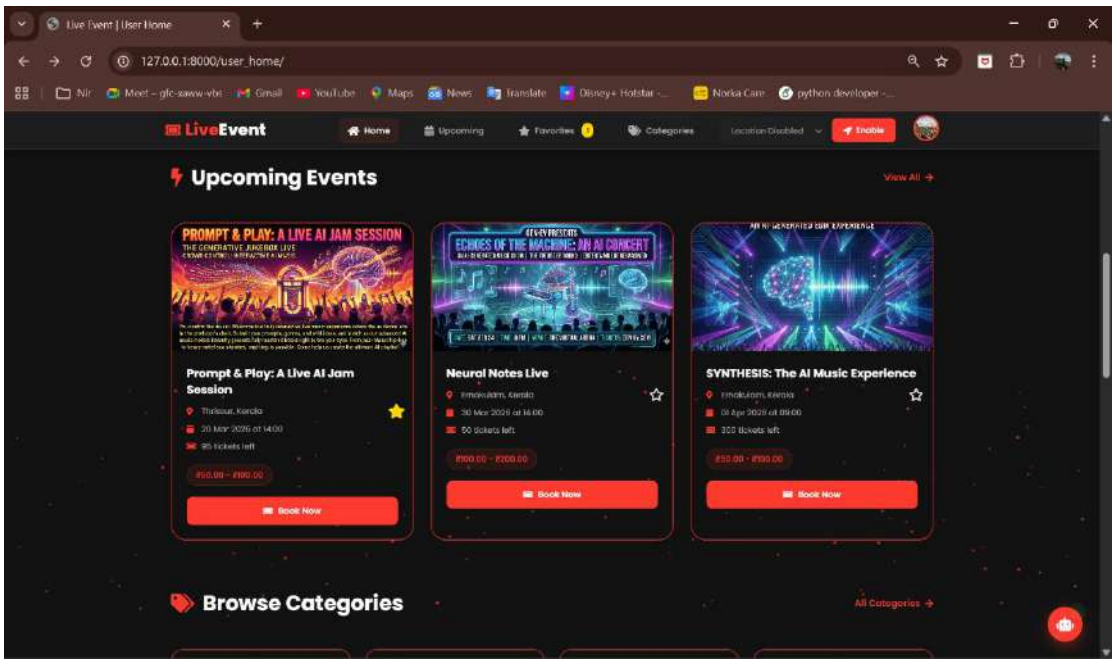
The outputs are designed in a simple and user-friendly format so that users can easily view and understand the information. Proper formatting and layout are used to ensure that the information is displayed clearly and efficiently.

The basic steps involved in output design are:

- Identify the required output information
- Select the appropriate output format
- Design the output layout
- Present the information clearly to the users

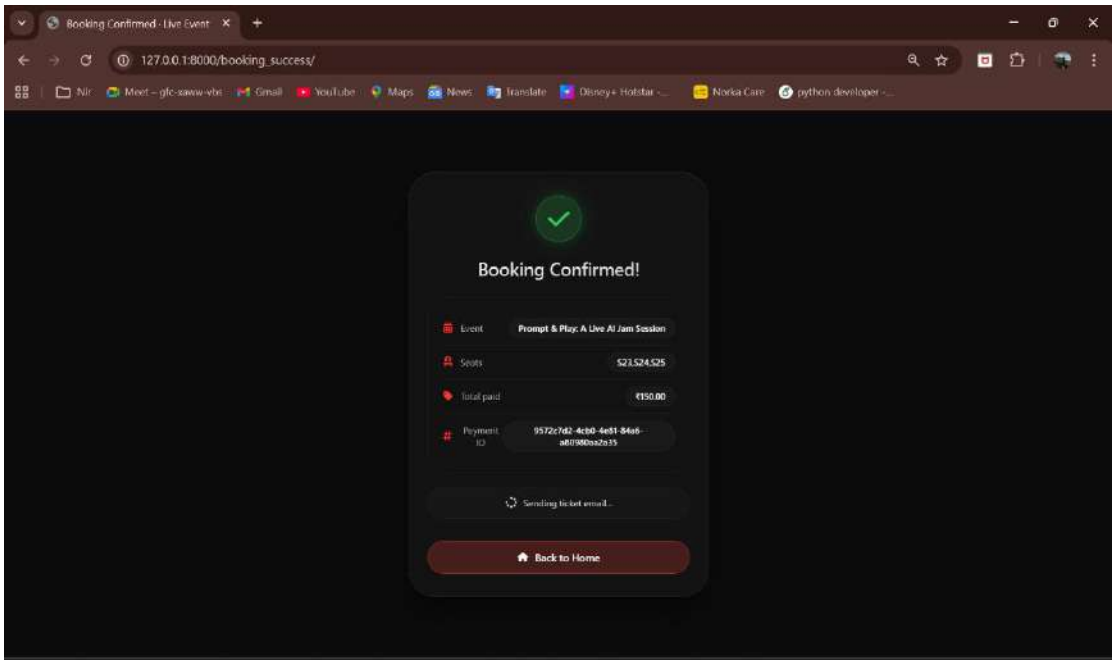
SCREENSHOTS

Event List Page

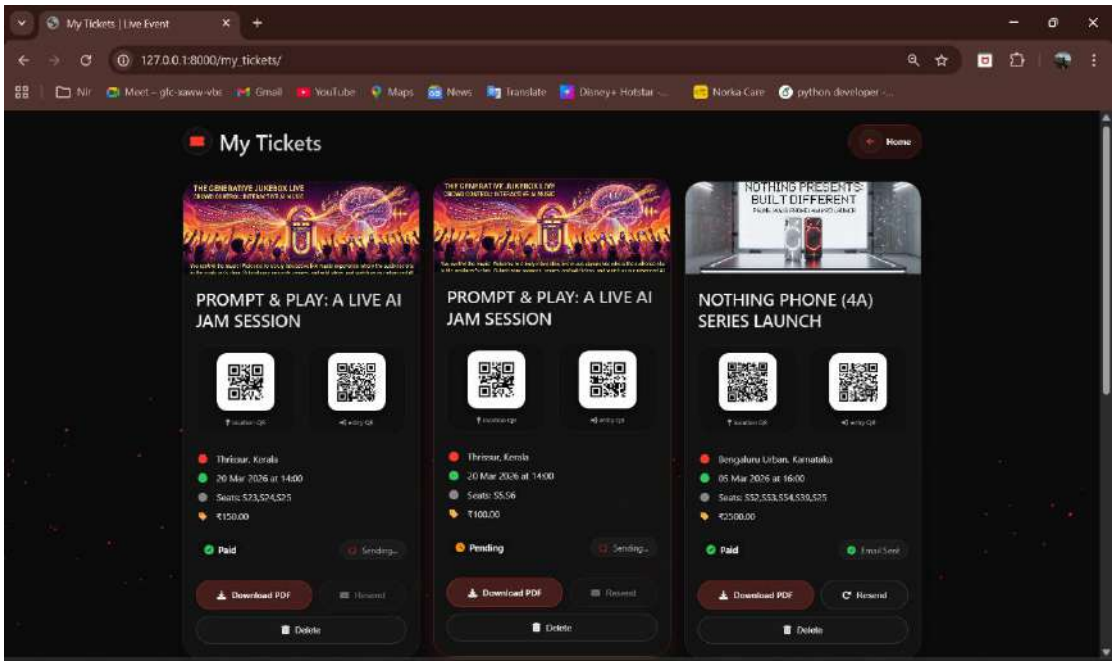


LiveEvent

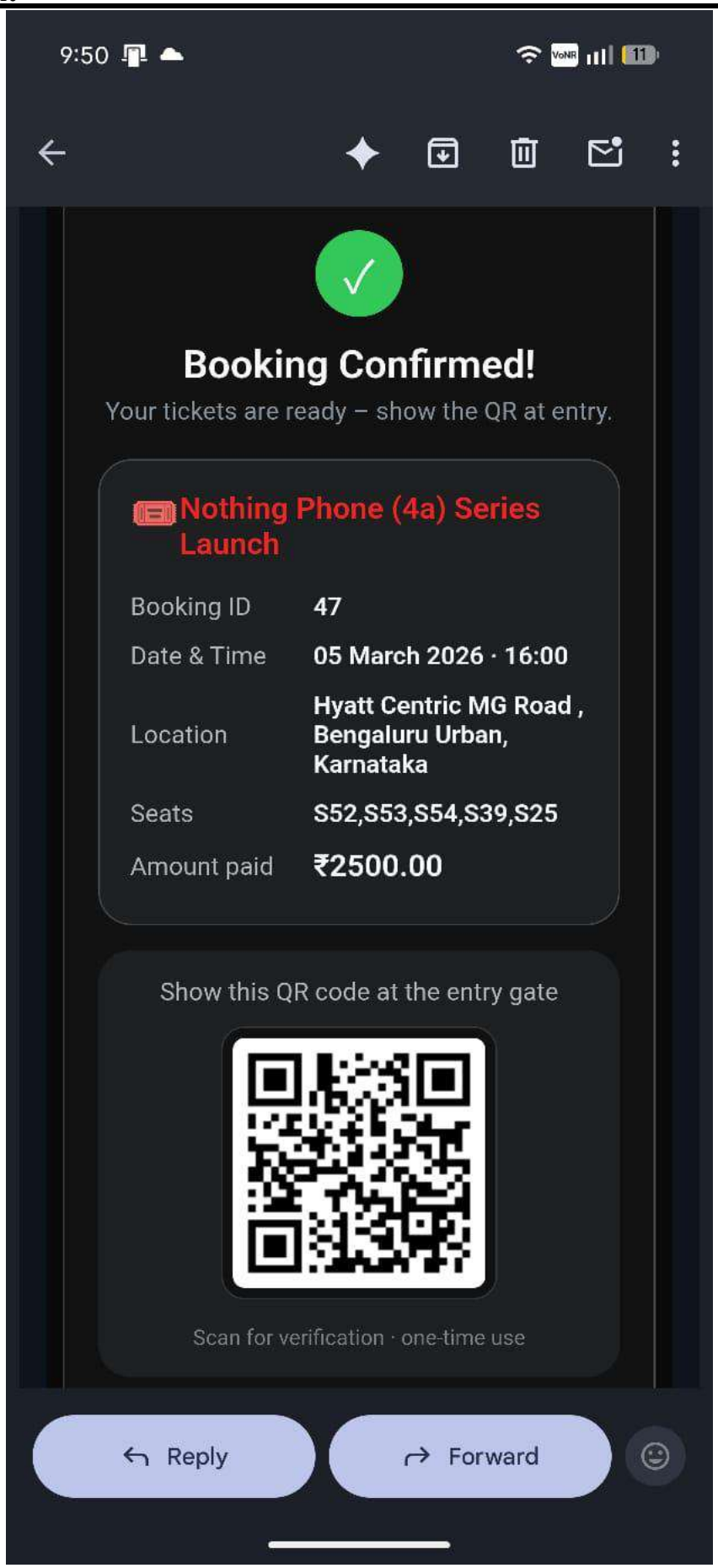
Booking Confirmation Page



Ticket QR Code

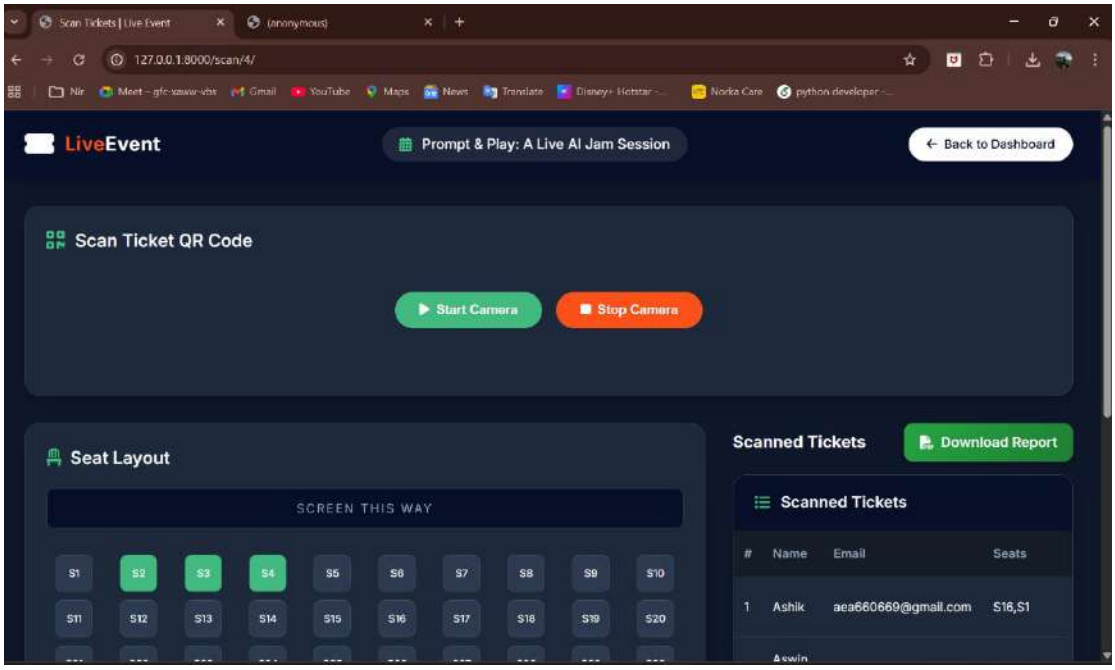


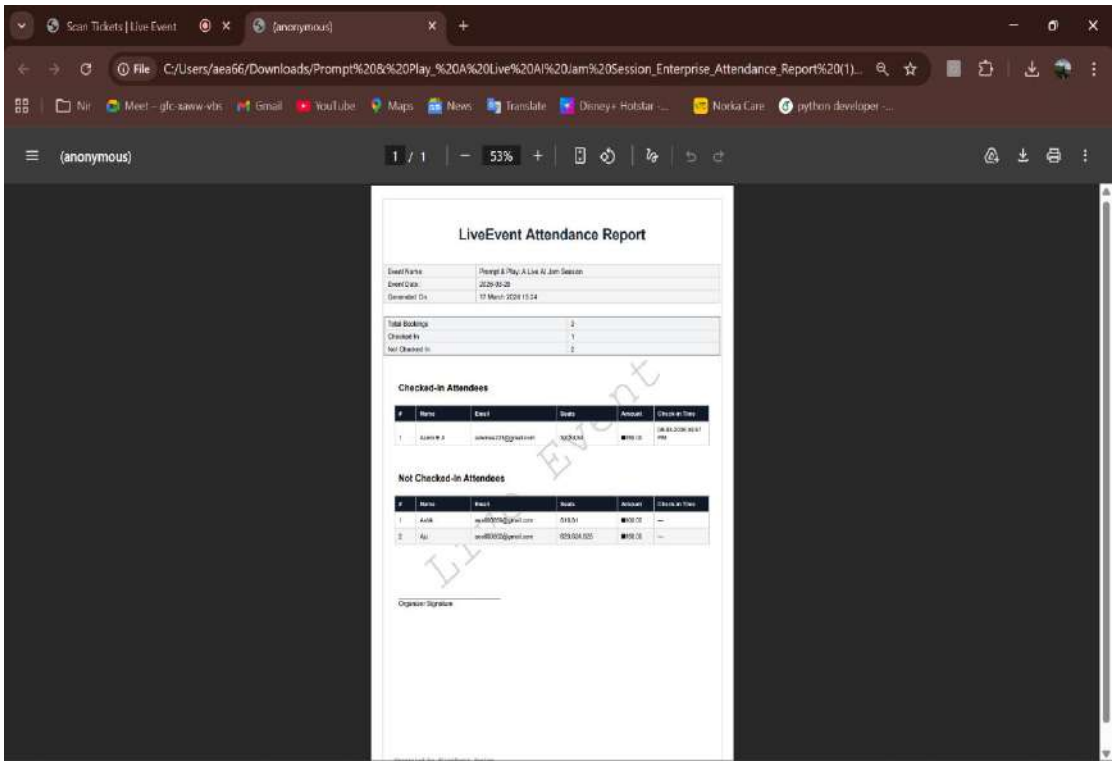
LiveEvent


9:50
Booking Confirmed!
Your tickets are ready – show the QR at entry.
Nothing Phone (4a) Series Launch
Booking ID 47
Date & Time 05 March 2026 · 16:00
Location Hyatt Centric MG Road , Bengaluru Urban, Karnataka
Seats S52,S53,S54,S39,S25
Amount paid ₹2500.00
Show this QR code at the entry gate
Scan for verification · one-time use
Reply
Forward

LiveEvent

□ Event Attendance Report

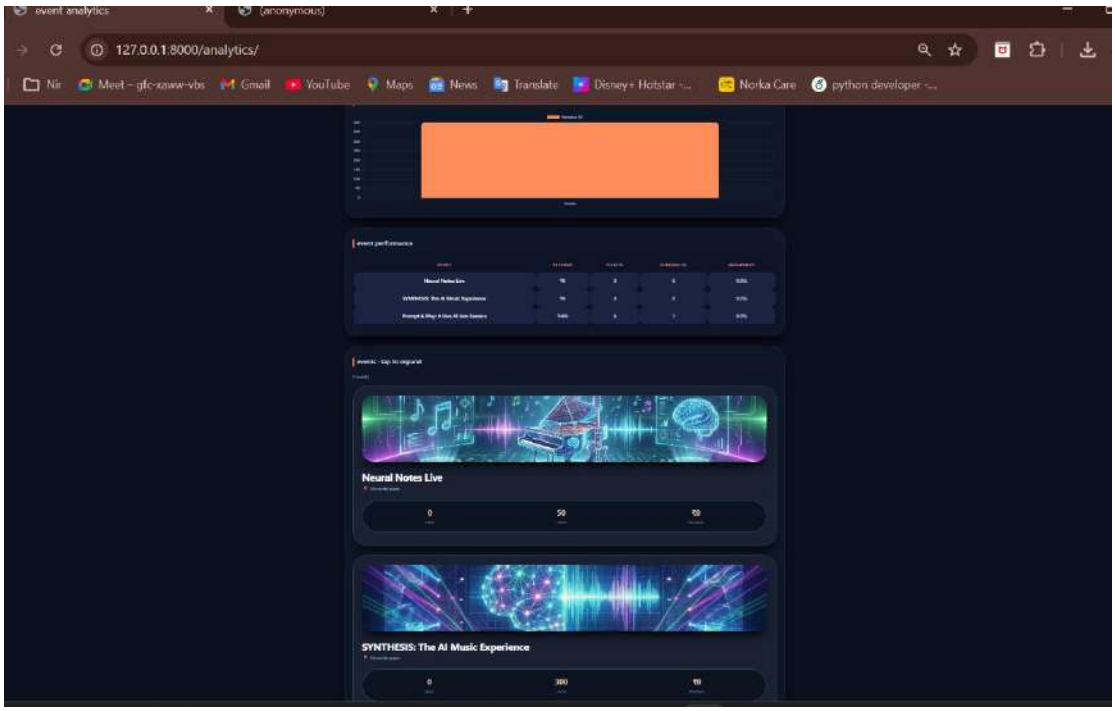




LiveEvent

Analytics Dashboard





Chapter 4 – System Environment

4.1 Introduction

The system environment describes the hardware and software platform required for developing and running the application. It includes the tools, technologies, and system requirements used to build and operate the system efficiently.

The **LiveEvent System** is developed as a web-based application using modern technologies. The system is implemented using the **Django framework with Python** as the backend programming language and **HTML, CSS, and JavaScript** for the frontend interface. The database used for storing system data is **SQLite**, which manages user information, event details, bookings, and feedback.

The system environment provides a reliable platform for the smooth functioning of the application. It ensures proper communication between the software components and allows users, organizers, and administrators to interact with the system effectively through a web browser.

4.2 Software Requirements Specification

Operating System: Windows 11

Web Framework: Django

Programming Language: Python 3.x

Web Browser: Google Chrome / Mozilla Firefox

Database: SQLite (db.sqlite3)

Front End Tool: HTML5, CSS, JavaScript

IDE: Visual Studio Code

4.3 Hardware Requirements Specification

Microprocessor: Intel(R) Core(TM)i5 13th gen

RAM: 16 GB

ROM: 1TB

Keyboard and Mouse: Standard

4.4 Tools And Platforms

4.4.1 Front End Tool

PYTHON

Python is an interpreted, object-oriented high-level programming language with dynamic semantics. Its high-level built-in data structures combined with dynamic typing and dynamic binding make it very attractive for rapid application development. Python's simple and easy-to-learn syntax emphasizes readability and reduces the cost of program maintenance.

Python supports modules and packages, which encourage program modularity and code reuse. The Python interpreter and standard library are available freely for major platforms and can be distributed without cost.

Python is widely used in many areas such as web development, machine learning, data analysis, automation, and desktop applications. In this project, Python is used as the primary programming language for developing the backend logic of the **LiveEvent System**.

Characteristics of Python

- Easy to learn
- Easy to read and maintain
- Large standard library
- Open-source and free
- Portable and platform independent

JAVASCRIPT

JavaScript is a programming language used to create interactive and dynamic web pages. It allows web applications to respond to user actions without reloading the page.

In the LiveEvent system, JavaScript is used to enhance user interaction, validate form inputs, and improve the responsiveness of the user interface. It helps create a more dynamic and user-friendly experience for the users.JavaScript works together with HTML and CSS to build modern web applications. It allows developers to create features such as dynamic content updates, interactive forms, and responsive layouts.

CASCADING STYLE SHEETS (CSS)

CSS (Cascading Style Sheets) is used to control the visual appearance and layout of web pages. It allows developers to separate the design of a website from its content.

CSS is used to style HTML elements such as text, colors, layouts, spacing, and positioning. This helps create visually attractive and user-friendly interfaces.

CSS also supports responsive design, allowing the website to adjust its layout depending on the screen size of the device being used.

Key Features of CSS

- Controls layout and design of web pages
- Separates design from content
- Supports responsive web design
- Improves user interface appearance

HTML

HTML (HyperText Markup Language) is the standard language used to create the structure of web pages. It defines the layout of content such as headings, paragraphs, images, links, and forms.

HTML elements act as the building blocks of web pages. Using HTML, developers can create structured documents that are displayed by web browsers.

In the LiveEvent system, HTML is used to design various pages such as registration forms, login pages, event listings, booking pages, and dashboards.

HTML works together with CSS and JavaScript to create complete web interfaces.

4.4.2 Back End Tool

DJANGO FRAMEWORK

Django is a high-level Python web framework that encourages rapid development and clean, practical design. It follows the **Model–View–Template (MVT)** architectural pattern, which helps developers organize application logic efficiently.

Django provides many built-in features such as authentication, database management, URL routing, and an administrative interface. These features allow developers to focus on application logic instead of writing repetitive code.

In the **LiveEvent System**, Django is used to handle the server-side functionality, including user authentication, event management, ticket booking, and database interaction. Django also provides security mechanisms that help protect the application from common web attacks such as SQL injection and cross-site scripting.

Features of Django

- Rapid development framework
- Secure and reliable
- Built-in admin panel
- Easy database integration
- Scalable and maintainable

CELERY

Celery is an asynchronous task queue used to handle background processes efficiently in web applications. It allows time-consuming tasks such as sending emails, generating QR codes, and processing data to be executed in the background without affecting the performance of the main application.

In the LiveEvent System, Celery is used to perform tasks such as sending confirmation emails, generating QR code tickets, and handling notifications asynchronously. This improves the responsiveness of the system and ensures that users do not experience delays while performing operations like booking tickets or registering for events.

Celery works with message brokers such as Redis to manage task queues and execute them in the background. By integrating Celery with Django, the system achieves better scalability, improved performance, and efficient task management.

Features of Celery:

- Supports asynchronous task execution
- Improves system performance and responsiveness
- Handles background jobs efficiently

- Scalable and distributed task processing
- Integrates easily with Django

SQLITE DATABASE

SQLite is a lightweight relational database management system used for storing application data. It is widely used in web applications because it is simple, efficient, and does not require a separate server for database management.

SQLite stores the entire database in a single file, making it easy to manage and maintain. It supports standard SQL queries and integrates seamlessly with the Django framework.

In the **LiveEvent System**, SQLite is used to store information such as user details, event information, booking records, ticket details, and feedback data. The database ensures that all information is stored securely and can be retrieved efficiently whenever required.

4.4.3 Operating System

WINDOWS

Windows 11 is the latest major release of Microsoft's Windows operating system, officially released on October 5, 2021. It is the successor to Windows 10 and provides a modern and improved computing experience with enhanced performance, security, and user interface design.

Windows 11 introduces several improvements such as a redesigned Start menu, a centered taskbar, improved window management features like Snap Layouts and Snap Groups, and an integrated Widgets panel for quick access to information. The operating system also replaces Internet Explorer with the Chromium-based Microsoft Edge browser and includes integration with Microsoft Teams for communication.

One of the key advantages of Windows 11 is its improved security architecture, which includes features such as hardware-based isolation, secure boot, and trusted platform module (TPM) support. These features help protect the system from various security threats and improve overall system reliability.

Windows 11 also provides better support for modern hardware and software technologies, making it suitable for application development and testing. It supports a wide range of development tools and programming environments.

In this project, **Windows 11** is used as the development platform for designing and implementing the **LiveEvent System**. It provides a stable environment for running development tools such as **Python, Django Framework, Visual Studio Code, and web browsers** required for developing and testing the web application.

Chapter 5 – System Implementation

5.1 Introduction

Implementation is the final stage of system development and it is an important phase in the software development process. It involves programming the system, testing the modules, training the users, and running the developed system in a real environment. During this phase the designed system is converted into a working application.

In the **LiveEvent System**, the implementation phase includes developing different modules such as user registration, event management, ticket booking, feedback system, and admin management. The system is developed using **Python and Django framework**, while **HTML, CSS, and JavaScript** are used for designing the user interface. The application uses **SQLite database** to store user information, event details, bookings, and feedback.

During implementation the system takes its physical form. Proper planning and development practices are followed to ensure that the system works efficiently and provides a user-friendly interface. The developed system is tested using sample data and necessary corrections are made whenever errors are detected. The implementation phase finally ends with evaluation of the system after running it for a certain period.

5.2 Coding

Coding is the process of converting the system design into executable program code. It is undertaken once the design phase is completed and the design documents have been reviewed successfully. Coding plays an important role in software development because it transforms the system requirements into a functional application.

In the LiveEvent system, coding is performed using Python programming language with Django framework. Different modules identified in the design phase are coded according to their specifications. These modules include user management, event creation, ticket booking, feedback management, and admin control.

During the coding phase, each module is developed independently and tested to ensure correct functionality. Proper programming practices are followed to maintain readability and maintainability of the code.

The system also utilizes Celery for handling background tasks such as sending confirmation emails, generating QR code tickets, and managing asynchronous operations. This improves the performance of the application by ensuring that time-consuming tasks do not affect user experience.

- The input to the coding phase is the system design document.
- Each module specified in the design document is coded separately.
- Modules are unit tested to ensure correct functionality.
- Coding standards are followed for maintaining code quality.

5.2.1 Coding Standards

Coding standards are a set of guidelines followed while writing program code to ensure consistency, readability, and maintainability. These standards help developers understand the code easily and make future modifications easier.

In this project, coding standards are followed while developing the LiveEvent system using Python and Django. Proper naming conventions, indentation, and documentation are maintained throughout the program.

Purpose of Having Coding Standards

- Coding standards give a uniform appearance to the program code.
- They improve readability and maintainability of the system.
- They reduce complexity and help detect errors easily.
- They encourage good programming practices.
- They improve efficiency of the programmers.

5.2.2 Sample Codes

The LiveEvent System is implemented using Python and Django framework. The following sample code shows a simple model used for storing user information in the database.

```
from .models import Feedback
from django.utils.timezone import localtime


def index(request):
```

LiveEvent

```
    upcoming_events = (
        EventCreate.objects
        .filter(
            event_date__gte=date.today(),
            created_by__is_blocked=False
        )
        .select_related('eventlocation', 'eventticket', 'created_by')
        .order_by('event_date')
    )

    return render(request, 'index.html', {
        'upcoming_events': upcoming_events
    })

import re

def strong_password(password):
    return (
        len(password) >= 6 and
        re.search(r"[A-Z]", password) and
        re.search(r"[a-z]", password) and
        re.search(r"[0-9]", password) and
        re.search(r"[^A-Za-z0-9]", password)
    )

def test_email(request):

    subject = " 🎉 LiveEvent Email Test"

    html_content = """
    <div style="font-family:Arial;padding:20px;background:#0f1525;">
        <div style="max-width:500px;margin:auto;background:white;
                    padding:30px;border-radius:12px;
```

```
                box-shadow:0 5px 15px rgba(0,0,0,0.3);">

        <h2 style="color:#4e73df;">LiveEvent Email Working ✅</h2>

        <p>Your email configuration is successfully set up.</p>

        <p style="margin-top:20px;font-size:12px;color:gray;">
          © 2025 LiveEvent
        </p>
      </div>
    </div>
    """

    text_content = strip_tags(html_content)

    email = EmailMultiAlternatives(
        subject,
        text_content,
        settings.DEFAULT_FROM_EMAIL,
        ["ashikaswin1@gmail.com"],
    )

    email.attach_alternative(html_content, "text/html")
    email.send()

    return HttpResponse("Professional test email sent successfully 🚀")
@never_cache
def logout(request):
    messages.success(request, "Logged out successfully 👋")
    request.session.flush()  # completely destroy session
    return redirect('index')
```

```
def user_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.session.get("email") or request.session.get("role") != "user":
            return redirect("index")
        return view_func(request, *args, **kwargs)
    return wrapper
@never_cache
@user_required
def user_home(request):

    email = request.session.get("email")
    user = user_register.objects.get(email=email)

    today = date.today()

    # ⭐ FAVORITES
    favorites = Favorite.objects.filter(user=user)
    favorite_ids = list(favorites.values_list("event_id", flat=True))
    favorite_count = favorites.count()

    user_favorites = favorites.select_related("event")
    favorite_events = [fav.event for fav in user_favorites]
```

```
# ⭐ UPCOMING EVENTS
upcoming_events = (
    EventCreate.objects
    .filter(
        event_date__gte=today,
        created_by__is_blocked=False  # 🔥 Only show active events
    )
    .select_related("eventlocation", "eventticket", "created_by")
    .order_by("event_date")
)

# ---------- FILTERS ----------
state = request.GET.get("state")
district = request.GET.get("district")
main_category = request.GET.get("main_category")

if state:
    upcoming_events = upcoming_events.filter(
        eventlocation__state=state
    )

if district:
    upcoming_events = upcoming_events.filter(
```

```
            eventlocation__district=district
        )

    if main_category:
        upcoming_events = upcoming_events.filter(
            created_by__main_category=main_category
        )

    # ---------- CALCULATE REMAINING TICKETS FOR UPCOMING EVENTS ----------
    for event in upcoming_events:

        try:
            total_limit = event.eventticket.tickets_limit or 0
        except:
            total_limit = 0

        paid_bookings = EventBooking.objects.filter(
            event=event,
            payment_status="Paid"
        )

        sold = 0
        for booking in paid_bookings:
            if booking.selected_seats:
```

LiveEvent

```
            sold += len(booking.selected_seats.split(","))


    event.remaining_tickets = max(total_limit - sold, 0)


  # ---------- CALCULATE REMAINING TICKETS FOR FAVORITE EVENTS ----------
  for event in favorite_events:


    try:
      total_limit = event.eventticket.tickets_limit or 0
    except:
      total_limit = 0


    paid_bookings = EventBooking.objects.filter(
      event=event,
      payment_status="Paid"
    )


    sold = 0
    for booking in paid_bookings:
      if booking.selected_seats:
        sold += len(booking.selected_seats.split(","))


    event.remaining_tickets = max(total_limit - sold, 0)
```

```
    return render(request, "user_home.html", {
        "user": user,
        "upcoming_events": upcoming_events,
        "selected_state": state,
        "selected_district": district,
        "selected_main_category": main_category,
        "favorite_events": favorite_events,
        "favorite_ids": favorite_ids,
        "favorite_count": favorite_count,
    })
```

5.3 Unit Testing

Unit testing involves testing each individual unit or component of the software application. It is the first level of functional testing and is performed during the development phase of the software. The main aim of unit testing is to validate that each component of the system works correctly and performs its intended function.

A unit is the smallest testable part of a software system. In object-oriented programming, a unit may be a method, class, or module. In procedural programming, a unit may be a function or procedure. Unit testing helps developers detect errors early in the development cycle and improves the overall quality of the software.

In the **LiveEvent System**, unit testing is performed on various modules such as user registration, user login, event creation, ticket booking, and feedback submission. Each module is tested independently using different sets of input data to verify that it produces the expected output.

Unit testing helps developers understand the code structure and ensures that individual components function correctly before integrating them with other parts of the system. The major advantage of unit testing is that it improves code quality and reduces the time and effort required for debugging later stages of development.

5.3.1 Test Plan

A test plan is a document that describes the testing strategy, scope, resources, and schedule of testing activities for a software project. It acts as a guide for the testing process and helps ensure that all components of the software are tested properly before the system is deployed. A well-defined test plan helps identify errors early in the development cycle and improves the quality and reliability of the system.

In the **LiveEvent System**, the test plan is designed to verify that all modules of the application work correctly and meet the specified system requirements. The testing process ensures that the system performs correctly under different conditions and that both valid and invalid inputs are handled properly.

The test plan also ensures that the system provides accurate outputs, maintains data integrity, and performs efficiently during operation. Testing is carried out at different levels such as unit testing, integration testing, and system testing to ensure the proper functioning of the application.

The following components are included in the test plan:

1. Test Objectives

The objective of testing is to verify that the system functions according to the specified requirements and design. It ensures that the software operates correctly and provides accurate results. The main objectives of testing are:

- To verify that each module works correctly.
- To identify errors and defects in the system.
- To ensure that the system meets user requirements.
- To check the interaction between different modules.
- To ensure the reliability and stability of the application.

2. Scope

The scope defines the modules and functionalities that will be tested during the testing process. In the LiveEvent system, the following modules are included in the testing scope:

- User Registration
- User Login

- Event Creation and Management
- Ticket Booking
- QR Code Ticket Generation
- Feedback Submission
- Admin Management

Testing ensures that these modules perform correctly and interact properly with the database and other system components.

3. Test Strategy

The test strategy describes the approach used for testing the system. Different levels of testing are applied to ensure the correctness of the system.

The testing strategy includes:

- **Unit Testing** – Testing individual modules separately.
- **Integration Testing** – Testing the interaction between modules.
- **System Testing** – Testing the complete system as a whole.
- **Functional Testing** – Verifying that each function works as expected.
- **Validation Testing** – Ensuring that the system meets user requirements.

4. Test Environment

The test environment refers to the hardware and software configuration required for testing the application. The environment used for testing the LiveEvent system includes:

Operating System: Windows 11
Programming Language: Python
Framework: Django
Database: SQLite
Web Browser: Google Chrome / Mozilla Firefox
Development Tool: Visual Studio Code

This environment is configured to simulate real user interaction with the system.

5. Test Deliverables

Test deliverables are the documents and outputs generated during the testing process. These deliverables help in monitoring the testing activities and tracking system defects.

The major test deliverables include:

- Test Plan Document
- Test Case Documents
- Test Execution Reports
- Bug or Defect Reports
- Test Summary Report

These documents provide detailed information about the testing process and results.

6. Test Schedule

The test schedule defines the timeline for performing testing activities. It includes planning, execution, and evaluation of the testing process.

The schedule ensures that testing is completed within the planned time frame and includes:

- Preparation of test plan
- Development of test cases
- Execution of test cases
- Identification and correction of errors
- Final verification of the system

7. Risks and Contingencies

Risks are possible issues that may affect the testing process or system performance. Identifying risks helps in preparing solutions in advance.

Possible risks include:

- Software compatibility issues
- Unexpected system errors
- Data inconsistency problems
- Hardware or network failures

To reduce these risks, proper backup strategies, error handling mechanisms, and regular testing procedures are implemented.

5.3.2 Test Cases

Test Case Id: Test 1

Correct Data:

Username: user@gmail.com

Password: user123

Function: User Login

Table 5.1 – User Login

SI NO	STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Enter username only	Username: user@gmail.com	Please enter username and password	Please enter username and password	success
2	Enter incorrect username and password	Username: user@gmail.com Password: wrong123	Wrong username and password	Wrong username and password	success
3	Enter username and password correctly	Username: user@gmail.com Password: user123	Login successfully	Login successfully	success

Test Case Id: Test 2

Correct Data:

Username: ashik

Email: ashik@gmail.com

Phone: 9876543210

Password: ashik123

Function: User Registration

Table 5.2 – User Registration

SI NO	STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Enter correct registration details	Registration information	Registered successfully	Registered successfully	success
2	Enter email without @ symbol	Email: ashikgmail.com	Invalid email	Invalid email	success
3	Enter phone number less than 10 digits	Phone: 7418	Invalid phone number	Invalid phone number	success
4	Enter password less than required characters	Password: ash	Enter password with minimum length	Enter password with minimum length	success

Test Case Id: Test 3

Correct Data:

Event Name: Tech Fest

Function: Event Creation

Table 5.3 – Event Creation

SI NO	STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Enter correct event details	Event Name: Tech Fest	Event created successfully	Event created successfully	success
2	Leave event name empty	Event Name: blank	Enter event name	Enter event name	success

Test Case Id: Test 4

Correct Data:

Event: Music Night

Tickets: 2

Function: Ticket Booking

Table 5.4 – Ticket Booking

SI NO	STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Select event and book ticket	Event: Music Night	Ticket booked successfully	Ticket booked successfully	success
2	Book without selecting event	Event: none	Please select event	Please select event	success

Chapter 6 – System Testing

6.1 Introduction

The purpose of system testing is to identify and correct errors in the developed system. Testing is an important element of software quality assurance and represents the final review of the system specification, design, and coding. As software systems become more complex, the cost associated with software failure increases, making well-planned testing essential.

Software testing is a critical phase in the software development process. Once the source code has been generated, the program should be executed before it is delivered to the user with the intention of finding and correcting errors. Testing techniques provide systematic guidance for designing test cases and evaluating system performance.

In the **LiveEvent System**, testing is performed to ensure that the system functions correctly and meets the specified requirements. The system is tested using various test cases and different sets of input data to verify that the system behaves as expected under different conditions.

To uncover errors in the program behaviour, functionality, and performance, the following steps are performed:

• Execute the internal logic of the software components.
• Execute the input and output domains of the program to identify errors.
• Use the system experimentally to ensure that it runs according to the specifications and user expectations.
• Prepare test data carefully to test the system under different conditions.
• Identify and correct errors using appropriate testing techniques.

System testing is conducted to detect errors and to compare the developed system with the requirement specifications. During testing, the software is executed using a set of test cases and the outputs are evaluated to determine whether the system performs as expected.

Testing presents an important challenge for software engineers because it ensures that the system works correctly in real-world conditions. Testing is the process of executing a program with the objective of finding errors. A good test case is one that has a high probability of detecting previously undiscovered errors.

Software reliability refers to the probability that the software will perform its required functions without failure for a specified period under specified conditions. A failure occurs when the system or one of its components is unable to perform its required function.

Different levels of testing are applied to ensure that the software system becomes reliable, error-free, and efficient. The major types of testing used in the system include:

- Unit Testing
- Integration Testing
- System Testing

6.2 Integration Testing

Integration testing is the process of combining individual modules that have already been unit tested and testing them together as a group. This level of testing ensures that the interaction between different modules of the system works correctly. The main objective of integration testing is to verify that the interfaces between modules function properly and that the data flows correctly from one module to another.

In the **LiveEvent System**, different modules such as user registration, login authentication, event management, ticket booking, feedback submission, and admin management are integrated and tested together. Each module interacts with the database and other system components, so it is important to verify that these modules communicate properly.

During integration testing, the system checks whether the data entered in one module is correctly reflected in another module. For example, when a user registers in the system, the information should be stored in the database and should be available for login authentication. Similarly, when a user books an event ticket, the booking details must be updated in the event management module and reflected in the system reports.

Integration testing ensures that the system functions smoothly when multiple modules work together and that the overall workflow of the system operates according to the design specifications.

6.3 System Testing

System testing is the final stage of the testing process where the complete and integrated system is tested as a whole. After designing and developing the software, all modules and subsystems are combined and tested to verify that the entire system works correctly.

In theory, each module of the system should function properly after unit and integration testing. However, in practice, when these modules are combined together, unexpected errors may occur. Therefore, system testing is performed to evaluate the system's behaviour when all components work together in a real operating environment.

In the **LiveEvent System**, system testing is conducted to ensure that the entire application functions according to user requirements. The system is tested to verify operations such as user registration, login, event creation, ticket booking, QR code generation, feedback submission, and administrative management.

The purpose of system testing is to ensure that the application performs correctly under different conditions and handles various types of inputs effectively. During this phase, the system is tested with different test cases to check the functionality, performance, and reliability of the application.

System testing represents the final opportunity to detect and correct errors before the system is deployed for user acceptance testing. This testing process ensures that the system meets the required specifications and performs efficiently when used by end users.

System testing also ensures that the application performs efficiently in a real-time environment by evaluating its performance, security, and usability aspects. The LiveEvent System is tested with different user inputs and scenarios to verify that it responds correctly and maintains data integrity throughout the process. Special attention is given to validating user interactions such as form submissions, navigation between pages, and database operations to ensure smooth functioning. The system is also checked for proper error handling and recovery mechanisms so that it can manage unexpected situations without failure. Overall, system testing confirms that the application is stable, reliable, and ready for deployment, providing a seamless experience for users, event organizers, and administrators.

6.3.1 Test Plan & Test Cases

Table 6.1 – Test Case (User Login)

Sl No	Test Steps	Expected Result	Actual Result	Pass/Fail
1	Run the application and navigate to the login page.	Login page should contain fields for entering username and password along with a login button.	Login page contains username field, password field, and login button.	Pass
2	Enter a valid username and password and press the login button.	User should successfully login and navigate to the dashboard.	Login successful and user redirected to dashboard page.	Pass
3	Enter a valid username and invalid password and press the login button.	A message should be displayed indicating invalid password.	Error message displayed for incorrect password.	Pass
4	Enter a valid username and leave password field empty, then press login button.	A message should be displayed asking to enter password.	Message displayed requesting password entry.	Pass
5	Press login button without entering username and password.	A message should be displayed that required fields must be filled.	Validation message displayed for required fields.	Pass
6	Enter invalid characters in username and password fields and press login button.	A validation message should be displayed indicating invalid input format.	Validation message displayed for incorrect input format.	Pass

Chapter 7 – System Maintenance

7.1 Introduction

Software maintenance is the process of modifying a software product after it has been delivered to the customer. The main purpose of software maintenance is to modify and update software applications after delivery to correct faults and to improve performance.

Software maintenance must be performed in order to correct faults, improve the design, implement enhancements, interface with other systems, accommodate programs so that different hardware, software, system features and telecommunication facilities can be used.

System maintenance is an umbrella term that encompasses various forms of computer maintenance needed to keep a system running. The two main components of system maintenance are preventive and corrective maintenance. Preventive maintenance involves taking measures to help keep the system functioning, whereas corrective maintenance involves the replacement or repair of a system or its components after they have already failed.

7.2 Maintenance

Maintenance projects can be used for enhancement or defect fixing. Enhancement projects are related to implementing new developments. It is basically modifications of software after it has been delivered in order to correct errors, to improve performance or other properties of IT systems. The key problems of software maintenance in IT systems include both management and technical problems. Important management issues are: adjusting to customer priorities, appropriate system maintenance personnel, cost estimation. The key technical issues are limited understanding, impact analysis, testing, measuring system maintainability, and infrastructure maintenance. It is well known that the guarantee of business survival is its continuous development and improvement of solutions, and thus also the continuous and adequate development of IT systems, combined with maintenance work carried out regularly on all components of the production environment.

CORRECTIVE MAINTENANCE

This refers to modifications initiated by defects in the software. A defect can result from design errors, logic errors and coding errors. Design errors occur when changes made to software are incorrect, incomplete, wrongly communicated or the change request is misunderstood. Logic errors result from invalid tests and conclusions, incorrect implementation of design specifications, faulty logic flow or incomplete test data. Coding errors are caused by implementation of detailed logic design and incorrect use of the source code logic. Defects are also caused by data processing errors and system performance errors.

In the event of system failure due to an error, actions are taken to restore the operation of the software system. Due to pressure from management, maintenance personnel sometimes resort to emergency fixes known as "patching". The adhoc nature of this approach often gives rise to a range of problems that include increased program complexity and unforeseen ripple effects. Unforeseen ripple effects imply that a change to one part of a program may affect other sections in an unpredictable manner, thereby leading to distortion in the logic of the system. This is often due to lack of time to carry out through "impact analysis" before effecting the change.

PREVENTIVE MAINTENANCE

Preventive maintenance (PM) is a proactive approach to maintenance that involves regularly scheduled inspections, servicing, and repairs of equipment or systems to prevent potential failures and to ensure they continue to operate efficiently. The main goal of preventive maintenance is to reduce the likelihood of unexpected breakdowns, minimize downtime, and extend the lifespan of the equipment. It is a planned and systematic strategy that focuses on keeping assets in good working condition, thus preventing costly repairs and production disruptions.

Key Characteristics of Preventive Maintenance:

- Scheduled Inspections: Preventive maintenance involves creating a maintenance schedule based on the manufacturer's recommendations, industry best practices, and historical data. Regular inspections are carried out to identify any signs of wear, damage, or deterioration.

- Routine Maintenance Tasks: During preventive maintenance, routine tasks are performed, such as cleaning, lubricating, tightening connections, and replacing worn-out parts. These activities help keep the equipment in optimal working condition.
- Predictable Costs: Since preventive maintenance is planned in advance, the costs associated with maintenance activities can be budgeted and accounted for, making it easier to manage expenses. Increased Reliability: By addressing potential issues before they escalate, preventive maintenance helps improve the overall reliability and performance of the equipment, reducing the risk of unexpected breakdowns.
- Safety Enhancement: Preventive maintenance can identify safety concerns and address them promptly, ensuring a safe working environment for employees and users of the equipment.

PERFECTIVE MAINTENANCE

Perfective maintenance, also known as "enhancement maintenance" or "adaptive maintenance," is a type of software maintenance that focuses on improving or enhancing the functionality, performance, and usability of a software system. Unlike corrective maintenance, which addresses issues and defects, perfective maintenance aims to make the software better, more efficient, and aligned with changing user needs or business requirements. It involves modifying the software to add new features, optimize performance, and enhance the user experience without changing its original functionality.

Key Objectives of Perfective Maintenance:

Feature Addition: Perfective maintenance involves adding new features, functionalities, or capabilities to the software to enhance its usefulness and extend its capabilities. These additions are based on user feedback, market trends, or business requirements.

Performance Optimization: The maintenance process may include re-engineering or optimizing the software to improve its speed, efficiency, and resource utilization. This can lead to a more responsive and faster-performing application.

Usability Improvement: The user interface and overall user experience can be enhanced to make the software more user-friendly and intuitive. This could involve redesigning elements, streamlining workflows, or adding helpful tooltips.

Code Refactoring: The maintenance team may refactor the codebase to improve its maintainability and readability without altering its external behaviour. This helps reduce technical debt and makes future enhancements and maintenance easier.

Compliance and Standards: Perfective maintenance may involve updating the software to comply with new industry standards, security regulations, or legal requirements.

ADAPTIVE MAINTENANCE

Adaptive maintenance is a type of software maintenance that focuses on modifying a software system to keep it functional and up-to-date in response to changes in the computing environment or to ensure its compatibility with evolving hardware, software, or external interfaces. The primary goal of adaptive maintenance is to make the software adapt and remain operational in changing circumstances without altering its original functionality or introducing new features.

Key Objectives of Adaptive Maintenance:

- Compatibility: Adaptive maintenance addresses issues that arise due to changes in the computing environment, such as operating system updates, changes in hardware configurations, or new software dependencies.
- External Interface Conformity: The maintenance process ensures that the software remains compatible with external systems and services it interacts with, such as databases, APIs, or web services.
- Regulatory Compliance: Adaptive maintenance may involve updating the software to meet new industry standards, data protection regulations, or legal requirements.
- Technology Upgrades: Upgrading underlying software libraries, frameworks, or programming languages may be necessary to ensure continued support and security updates.

Chapter 8 – System Security Measures

8.1 Introduction

The security of a computer system is a crucial task. It is a process of ensuring the confidentiality and integrity. Security is one of most important as well as the major task in order to keep all the threats or other malicious tasks or attacks or program away from the computer's software system. A system is said to be secure if its resources are used and accessed as intended under all the circumstances, but no system can guarantee absolute security from several of various malicious threats and unauthorized access.

The security of a system can be threatened via two violations:

Threat: A program that has the potential to cause serious damage to the system.

Attack: An attempt to break security and make unauthorized use of the assets.

Security violations affecting the system can be categorized as malicious and accidental threats. Malicious threats as the name suggests are a kind of harmful computer code or web script designed to create system vulnerabilities leading to back doors and security breaches. Accidental threats on the other hand, are comparatively easier to be protected.

Security can be compromised via any of the breaches mentioned:

- Breach of confidentiality: This type of violation involves the unauthorized reading of data.
- Breach of integrity: This violation involves unauthorized modification of data.
- Breach of availability: It involves unauthorized destruction of data.
- Theft of service: It involves the unauthorized use of resources.
- Denial of service: It involves preventing legitimate use of the system. As mentioned before, such attacks can be accidental in nature.

8.2 Database Level Security

Database security refers to the range of tools, controls, and measures designed to establish and preserve database confidentiality, integrity, and availability. Film casting only have a single admin. The chances of insider threat are too low. Validation is done in each registration thus the human error is handled well.

Here are some important database level security measures:

- **Authentication and Authorization:** Implement strong authentication mechanisms to verify the identity of users accessing the database. Enforce password policies, use two-factor authentication (2FA), and limit administrative privileges to authorized personnel. Assign appropriate user roles and permissions to control access to database objects and operations.
- **Data Encryption:** Use encryption techniques to protect sensitive data stored in the database. Encrypting data at rest and in transit helps prevent unauthorized access even if the underlying storage or network infrastructure is compromised. Implement transparent data encryption (TDE) or field-level encryption for sensitive data elements.
- **Access Controls:** Implement access controls at the database level to restrict unauthorized access to data. Use role-based access control (RBAC) or attribute-based access control (ABAC) mechanisms to enforce granular permissions and ensure that users can only access the data they are authorized to see.
- **Auditing and Logging:** Enable auditing and logging features provided by the database system to track and monitor database activities. Record user actions, schema changes, data modifications, and access attempts. Regularly review and analyze logs to detect any unusual or suspicious activities.

8.3 System-Level Security

System-level security refers to the architecture, policy and processes that ensure data and system security on individual computer systems. It facilitates the security of stand-alone and/or network computer system servers from events and processes that can exploit or violate its security or stature. It ensures that system access is granted only to legitimate and trusted individuals and applications. The key objective behind system-level security is to keep system secure, regardless of security policies and processes at other levels. If other layers or levels are breached, the system must have the ability to protect itself.

Here are some important system level security measures:

LiveEvent

- **Secure System Configuration:** Ensure that the system is securely configured by following security best practices and vendor recommendations. This includes disabling unnecessary services and features, removing default accounts and passwords, and implementing secure settings for network interfaces, firewalls, and other system components.
- **Regular System Updates and Patching:** Keep the system up to date with the latest security patches and updates provided by the software and hardware vendors. Regularly apply patches to address known vulnerabilities and security weaknesses. Enable automatic updates whenever possible.
- **Malware Protection:** Install reputable antivirus and anti-malware software on the system to detect and remove known malware threats. Keep the software updated with the latest virus definitions to ensure effective protection. Use behaviour-based detection and real-time scanning to detect and prevent new and emerging malware threats.
- **Host-Based Firewalls:** Configure host-based firewalls on the system to control inbound and outbound network traffic. Define rules to allow only necessary network connections and block unauthorized access attempts.
- **System Hardening:** Apply system hardening practices to minimize potential vulnerabilities. This includes disabling unnecessary services and protocols, removing or securing unnecessary user accounts, and applying security configurations and access controls to limit system exposure.

Chapter 9 – System Planning and Scheduling

9.1 Introduction

Planning and scheduling are distinct but closely related aspects of managing a successful project. Planning mainly focuses on selecting appropriate policies, procedures, and strategies required to achieve the objectives of the project. Scheduling, on the other hand, converts the project plan into a practical timetable that defines when and how project activities will be performed.

Planning defines the scope, time, cost, and quality requirements of a project, while scheduling organizes these requirements into a structured sequence of tasks. It also considers important factors such as human resources, communication, risk management, and resource allocation. Together, planning and scheduling provide an effective framework for managing and controlling project activities.

In the **LiveEvent System**, planning and scheduling played an important role in organizing the development process. The project plan defined the various stages of development such as requirement analysis, system design, coding, testing, and implementation. Scheduling helped allocate time for each stage of the project so that the development process could be completed within the specified timeline.

The project schedule acts as a major tool for monitoring and controlling project progress. It helps the development team track completed tasks, identify delays, and make necessary adjustments to ensure successful project completion. The integration of project planning and scheduling ensures that the project objectives are achieved efficiently throughout the system development life cycle.

9.2 Planning a Software Project

Project planning is an important phase in the project life cycle as it defines the objectives of the project and the methods used to achieve them. The planning phase involves documenting project requirements, defining deliverables, and preparing a development schedule. It helps guide the development team through the implementation and completion phases of the project.

During this phase, several plans are prepared to manage different aspects of the project such as time, cost, quality, risks, and resources. Proper planning ensures that the project is completed within the specified time frame and budget. It also helps coordinate the work of team members and manage external resources if required.

In the **LiveEvent System**, project planning helped organize the development activities such as requirement analysis, system design, coding, testing, and deployment. This planning ensured that the system was developed in a structured manner and met the specified project objectives.

The purpose of the project planning phase is to:

- Establish business and system requirements.
- Determine project cost, schedule, deliverables, and deadlines.
- Identify and allocate resources required for development.
- Obtain approval to proceed with the next phases of development.

The basic processes of project planning include:

- **Scope Planning** – Identifying the project scope and defining the system requirements.
- **Work Breakdown Structure Preparation** – Dividing the project into smaller tasks and activities.
- **Project Schedule Development** – Defining the sequence and timeline of project activities.
- **Resource Planning** – Determining the human and technical resources required for the project.
- **Budget Planning** – Estimating the cost required for completing the project.
- **Risk Management** – Identifying possible risks and preparing strategies to minimize them.
- **Quality Planning** – Ensuring that the system meets the required quality standards.

9.2.1 Steps Involved in Planning a System

The planning of a software system involves several steps that help organize the development process and ensure the successful completion of the project. One of the most widely used approaches for software development is the **Software Development Life Cycle (SDLC)**.

The **Software Development Life Cycle (SDLC)** is a structured process followed by software organizations to develop high-quality software systems. It provides a step-by-step approach for planning, designing, developing, testing, and maintaining software

applications. The SDLC helps improve the quality of the software and ensures that the project meets the specified requirements.

A typical SDLC consists of the following stages:

1. **Planning and Requirement Analysis**
   In this phase, the project requirements are gathered and analyzed. The objectives of the system and the needs of the users are identified.
2. **Defining Requirements**
   After gathering requirements, they are documented clearly so that the development team understands the system functionality and expected outcomes.
3. **Designing the Product Architecture**
   The system architecture and database structure are designed during this phase. It includes system design, interface design, and data modeling.
4. **Building or Developing the Product**
   In this phase, the system is developed based on the design specifications. The coding of different modules is carried out using appropriate programming languages and tools.
5. **Testing the Product**
   The developed system is tested to identify and correct errors. Different types of testing such as unit testing, integration testing, and system testing are performed.
6. **Deployment and Maintenance**
   After successful testing, the system is deployed for use. Maintenance activities are performed to fix issues, improve performance, and update the system when necessary.

PROJECT SCHEDULING

Project scheduling defines the timeline for completing different activities involved in the development of the system. It helps in organizing the development phases and ensures that each task is completed within the planned time.

Table 9.1 – Project Scheduling

TASK	START DATE	END DATE
System Study	20-11-2025	25-11-2025
Initial Report	26-11-2025	30-11-2025
Topic Submission	10-12-2025	10-12-2025
System Selection	12-12-2025	14-12-2025
Requirement Analysis	15-12-2025	25-12-2025
Technology Selection	26-12-2025	30-12-2025
Database Design	01-01-2026	10-01-2026
System Design	11-01-2026	25-01-2026
Coding / Implementation	26-01-2026	15-02-2026
Test Case Design	16-02-2026	20-02-2026
System Testing	21-02-2026	25-02-2026
System Updates / Finalization	26-02-2026	28-02-2026

Chapter 10 – Future Enhancement and Scope of Further Development

10.1 Introduction

Future enhancement refers to the possible improvements and additional features that can be added to the system in the future to improve its functionality, efficiency, and user experience. No software system is considered completely finished, as technology continues to evolve and new user requirements may arise over time.

The **LiveEvent System** has been developed to provide a platform for managing events, handling user registrations, booking tickets, and collecting feedback. Although the current system meets the basic requirements, there are several opportunities for further development and enhancement.

In the future, additional features such as online payment integration, mobile application support, advanced event analytics, and real-time notifications can be implemented. These enhancements will help improve the performance of the system and provide better services to users, event organizers, and administrators.

Future development may also include improvements in system security, scalability, and user interface design to ensure that the application can handle a larger number of users and events. By implementing these enhancements, the system can become more efficient, reliable, and adaptable to changing user needs.

10.2 Merits of the System

The **LiveEvent System** provides several benefits that help improve the overall event management process. The system reduces manual work and allows users and organizers to perform event-related activities efficiently through a single platform.

Some of the important merits of the system are:

- **Easy Event Management** – The system allows organizers to easily create, update, and manage events through an online platform.
- **Quick Ticket Booking** – Users can quickly book tickets for events without visiting the venue physically.
- **Centralized Data Storage** – All event details, user information, and booking records are stored in a centralized database for easy access and management.

• **Improved Accuracy** – Automation of processes reduces human errors that may occur in manual systems.

• **Better Accessibility** – Since the system is web-based, users can access it from anywhere using a web browser.

• **Time and Cost Efficient** – The system minimizes paperwork and manual processing, saving both time and operational costs.

• **Better User Experience** – The system provides an organized interface that makes it easier for users to interact with the application.

10.3 Limitations of the System

Despite the advantages provided by the LiveEvent System, there are certain limitations that exist in the current version of the system.

Some of the limitations include:

• **Dependence on Internet Connectivity** – Users must have internet access to use the system since it is a web-based application.

• **Limited Notification Features** – The current system may not support advanced notification features such as SMS or push notifications.

• **Basic Reporting Features** – The reporting and analytics capabilities of the system may require further improvement.

• **Limited Mobile Optimization** – The system may require additional improvements to fully support mobile devices and different screen sizes.

• **Future Security Enhancements** – Additional security mechanisms may be required to enhance protection against potential cyber threats.

10.4 Future Enhancement of the System

Future enhancements refer to the additional features and improvements that can be incorporated into the system to improve its performance, usability, and functionality. Although the **LiveEvent System** fulfills the current requirements, several enhancements can be implemented in the future to make the system more efficient and feature-rich.

Some possible future enhancements of the system include:

- **Online Payment Integration** – Adding secure online payment gateways such as credit card, debit card, or digital wallets for ticket booking.
- **Mobile Application Development** – Developing a mobile application for Android and iOS platforms to allow users to access the system more conveniently.
- **Real-Time Notifications** – Implementing email or SMS notifications to inform users about event updates, booking confirmations, and reminders.
- **Advanced Reporting and Analytics** – Providing detailed reports and analytics to help organizers understand event performance and user participation.
- **Enhanced Security Features** – Implementing additional security measures such as two-factor authentication and stronger encryption for user data.
- **Multi-User Support** – Allowing multiple organizers to manage events simultaneously with different levels of access control.

11 Annexure:

11.1 Organization Profile

The **LiveEvent System** was developed as an academic project as part of the **Master of Computer Applications (MCA)** curriculum. The project was developed independently by the student to apply theoretical knowledge gained during the course into a practical software application.

The project development was carried out using modern web technologies such as **Python, Django Framework, HTML, CSS, JavaScript, and SQLite database**. These technologies were used to design and implement a web-based platform that allows users to manage events, book tickets, and provide feedback.

The development of the system involved various stages of the **Software Development Life Cycle (SDLC)** including requirement analysis, system design, implementation, testing, and deployment. The project was developed using **Visual Studio Code** as the development environment on a **Windows operating system**.

This project helped enhance practical knowledge in areas such as web application development, database management, and system design. It also provided experience in developing a complete software system from planning to implementation.

11.2 Document Glossary, Figures, Tables

Document Glossary (Abbreviations)

Abbreviation	Full Form	Abbreviation
SDLC	Software Development Life Cycle	SDLC
ER Diagram	Entity Relationship Diagram	ER Diagram
UML	Unified Modeling Language	UML
UI	User Interface	UI
DBMS	Database Management System	DBMS
OTP	One Time Password	OTP

API	Application Programming Interface	API
QR Code	Quick Response Code	QR Code
HTML	HyperText Markup Language	HTML
CSS	Cascading Style Sheets	CSS
JS	JavaScript	JS

List of Figures

Figure No	**Title**	**Page No**
Fig 1	ER Diagram of the System	18
Fig 2	Use Case Diagram	28
Fig 3	Sequence Diagram	31
Fig 4	Activity Diagram	34,35
Fig 5	Organizer Register form	44
Fig 6	User Registration Page	43
Fig 7	Event Creation Page	46,47
Fig 8	Ticket Booking Page	48,49
Fig 9	Feedback Page	50
Fig 10	Booking Confirmation Page	52
Fig 11	Event Listing Page	51
Fig 12	QR Code Ticket	52,53

List of Tables

Table No	**Title**	**Page No**
Table 3.1	user_register Table	19
Table 3.2	EventRegister Table	20
Table 3.3	EventCreate Table	21
Table 3.4	Favorite Table	22
Table 3.5	EventLocation Table	22
Table 3.6	EventContact Table	23
Table 3.7	EventTicket Table	23
Table 3.8	EventBooking Table	24

Table 3.9	Feedback Table	25
Table 3.10	PasswordResetOTP Table	26
Table 5.1	User Login Test Case	71
Table 5.2	User Registration Test Case	72
Table 5.3	Event Creation Test Case	73
Table 5.4	Ticket Booking Test Case	73
Table 6.1	System Testing – User Login	78

11.3 References

11.3.1 Books

The following books were referred during the development of this project:

1. **Ian Sommerville**, *Software Engineering*, 10th Edition, Pearson Education Limited.
2. **Roger S. Pressman and Bruce R. Maxim**, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education.
3. **Paul Barry**, *Head First Python: A Brain-Friendly Guide*, 2nd Edition, O'Reilly Media.
4. **Mark Lutz**, *Learning Python*, 5th Edition, O'Reilly Media.
5. **Steven Holzner**, *HTML, CSS and JavaScript Programming*, McGraw-Hill Education.

11.3.2 Websites

The following websites were referred during the development of this project:

1. https://www.djangoproject.com/
   Date of Access: 15 March 2026
2. https://docs.python.org/3/
   Date of Access: 15 March 2026
3. https://developer.mozilla.org/
   Date of Access: 15 March 2026
4. https://www.w3schools.com/
   Date of Access: 15 March 2026
5. https://stackoverflow.com/
   Date of Access: 15 March 2026